ABC RETAIL DATA PLATFORM

SNOWFLAKE + DBT END-TO-END

Tutorial triển khai chuyên sâu

TỪ FILE ĐẾN DATA MART

Medallion | CDC | SCD2 | Dimensional Model

dbt | Time Travel | Sharing | Cortex AI

HANDBOOK THỰC HÀNH

Phiên bản 2.0 • Cập nhật 28/07/2026

MỤC LỤC

Cấu trúc tutorial được thiết kế theo đúng thứ tự triển khai.

0. Bắt đầu tại đây 6

1. Kết quả cuối cùng 6

1.1. Định nghĩa "hoàn thành" . . . 7

2. Công nghệ và nền tảng 7

2.1. Thành phần bắt buộc . . . 8

2.2. Thành phần tùy chọn . . . 8

2.3. Phạm vi edition và quyền . . . 8

3. Cài đặt và setup 8

3.1. Tạo Snowflake account . . . 8

3.2. Cài dbt Core cho Snowflake . . . 9

3.3. Cấu trúc repository . . . 10

3.4. Cách chạy SQL . . . 10

4. Bootstrap: role, database, schema và warehouse 10

4.1. Role model . . . 10

4.2. Script bootstrap . . . 11

Kiểm tra . . . 11

4.3. Resource Monitor . . . 11

5. Tạo dữ liệu mẫu 12

5.1. Bộ dữ liệu landing thực tế . . . 12

5.2. Ví dụ JSON . . . 12

5.3. Nguyên tắc sinh dữ liệu . . . 13

6. Kết nối S3, stage và file format 13

6.1. Xác định đúng S3 prefix . . . 13

6.2. Tạo IAM policy chỉ đọc cho landing . . . 13

6.3. Tạo IAM role . . . 14

6.4. Tạo Storage Integration . . . 14

6.5. Cập nhật trust policy và xác thực . . . 15

6.6. File formats và S3 external stage . . . 16

6.7. Tùy chọn Snowpipe auto-ingest . . . 16

6.8. Cấu hình dbt project . . . 17

6.9. Khai báo Bronze sources . . . 18
6.10. Ranh giới trách nhiệm . . . 18

7. Bronze layer: raw, append-only và lineage 18

7.1. Table design . . . 19
7.2. Macro ingestion và cách chạy . . . 19
7.3. Load JSON bằng raw-line ingestion . . . 20
7.4. Quarantine parse errors . . . 20
7.5. CSV vào raw VARIANT . . . 21
7.6. Load Products, Feedback và CDC tombstones . . . 21

8. Quarantine, audit và data quality 22

8.1. Quarantine table . . . 22
8.2. Audit table . . . 22
8.3. Rule catalog . . . 22

9. Silver layer: parse, type, deduplicate 22

9.1. Silver Events . . . 23
9.2. Silver Orders . . . 24
9.3. Silver Customer Changes . . . 25
9.4. FX rates và Products . . . 25
9.5. Customer Feedback . . . 27

10. CDC trên Snowflake: Streams 27

10.1. Hiểu đúng CDC . . . 28
10.2. Tạo streams . . . 28
10.3. Source delete tombstones . . . 28

11. Stored Procedure xử lý Orders 29

11.1. Chiến lược . . . 29
11.2. Procedure mẫu . . . 30

12. SCD Type 2 triệt để, kể cả late-arriving change 32

12.1. Tại sao MERGE đơn giản chưa đủ . . . 32
12.2. Key map để surrogate key ổn định . . . 32
12.3. Rebuild affected timelines . . . 34
12.4. Test SCD2 . . . 35

13. Dimensional modeling 35

13.1. Chọn grain trước khi viết DDL . . . 35

13.2. Star schema . . . 35
13.3. Snowflake schema . . . 36
13.4. Normalization và denormalization . . . 36

14. Load facts với point-in-time dimension lookup 36

14.1. Fact Order Line . . . 36

15. Gold data marts 37

15.1. Revenue Daily . . . 37
15.2. Daily Funnel . . . 38
15.3. Customer 360 . . . 38

16. Dynamic Tables 38

17. Tasks và Task Graph 39

17.0. Orchestration chính bằng dbt . . . 39
17.1. Root scheduled task . . . 39
17.2. Child tasks . . . 39
17.3. Triggered task thay cron . . . 40

18. Test và reconciliation 40

18.1. Idempotency . . . 40
18.2. Orphan fact . . . 40
18.3. Revenue reconciliation . . . 40

19. Unload data 40

20. Zero-Copy Cloning 41

21. Time Travel và Fail-safe 41

21.1. Query lịch sử . . . 41
21.2. Khôi phục update/delete sai . . . 41
21.3. UNDROP . . . 42
21.4. Fail-safe . . . 42

22. Secure Data Sharing 42

22.1. Secure view . . . 42
22.2. Provider . . . 42
22.3. Consumer . . . 42

23. Cortex AI 42

23.1. Quyền . . . 42

23.2. Enrichment table . . . 43
23.3. Incremental AI processing . . . 43
23.4. Đánh giá . . . 44

24. Performance và cost **44**

24.1. Query Profile . . . 44
24.2. Micro-partition pruning . . . 45
24.3. Cache-aware benchmark . . . 45
24.4. Cost guardrails . . . 45

25. Deployment và runbook **45**

25.1. Thứ tự deploy . . . 45
25.2. Quy trình chạy một batch . . . 46
25.3. Xử lý sự cố . . . 46
25.4. Recovery runbook . . . 46

26. Cleanup **46**

27. Checklist nghiệm thu **47**

Foundation . . . 47
Pipeline . . . 47
Modeling . . . 47
Platform . . . 47
Operations . . . 47

28. Tài liệu tham khảo chính thức **48**

29. Lời kết **49**

0. Bắt đầu tại đây

Day la thu tu thao tac chinh thuc. Neu ban moi lam project, **khong doc va chay tu dau den cuoi mot cach may moc**:

Giai doan	Muc can lam	Chay o dau	Ket qua bat buoc
1. Kiem tra	3.1 va 3.4	Snowsight	Biet account, role, warehouse
2. Bootstrap	Muc 4	Snowsight, role ACCOUNTADMIN	Role, WH, DB, schema
3. Hieu data	Muc 5	Chi doc + local terminal	Biet schema va edge cases
4. Ket noi S3	Muc 6.1-6.6	AWS Console + Snowsight	`LIST @STG_S3_LANDING` thanh cong
5. Khoi tao dbt	Muc 6.8-6.10	Local terminal	`dbt debug` thanh cong
6. Ingestion	Muc 7-8	`dbt run-operation`	Bronze, quarantine, audit
7. Transform	Muc 9-16	`dbt build`	Silver, DWH, marts
8. Orchestration	Muc 17	dbt job hoac Snowflake Task	Pipeline co schedule
9. Nghiem thu	Muc 18 va 27	dbt + Snowsight	Test va reconciliation pass
10. Demo platform	Muc 19-24	Snowsight	Clone, Time Travel, share, AI

Du lieu da co tren `s3://landing1310`. Vi vay:

- Khong sinh lai du lieu.
- Khong upload `landing/` bang Snowflake CLI.
- Khong bat buoc cai Snowflake CLI.
- dbt la cong cu chinh de quan ly transformation, test, lineage va deployment.
- Snowsight van bat buoc cho bootstrap quan tri, Storage Integration va cac demo platform can quyen account.

Lenh dieu khien hang ngay:

```
dbt debug
dbt run-operation load_all_bronze --args '{batch_id: manual_001}'
dbt build --select tag:silver+
dbt test
dbt docs generate
dbt docs serve
```

> Lan dau tien, dung sau moi giai doan va chay truy van xac nhan. Chi chuyen sang buoc tiep theo khi ket qua bat buoc trong bang tren da dat.

1. Kết quả cuối cùng

Sau khi hoan thanh, ban se co mot pipeline:

```
CSV / JSON / Parquet trong S3 bucket landing1310
        |
        v
S3 External Stage + Storage Integration
        |
        | COPY INTO
        v
BRONZE - append-only, VARIANT, metadata
        |
        | dbt staging/incremental models
        v
SILVER - typed, deduplicated, conformed
        |
        | dbt incremental + timeline model + tests
```

```
        v
CORE_DWH - dimensions + facts
        |
        | dbt models / Dynamic Table optional
        v
MART - funnel, revenue, customer 360, quality
        |
        +--> Secure Data Sharing
        +--> Cortex AI enrichment
        +--> BI / ad-hoc analytics
```

Ket qua khong chi la cac table. Ban se co:

- Phan quyen theo role va warehouse tach biet.
- Load file co validation, lineage, quarantine va audit.
- Incremental processing co retry an toan.
- SCD Type 1 cho order va SCD Type 2 cho customer.
- Star schema va snowflake schema de so sanh.
- Data marts co reconciliation.
- dbt DAG, tests, documentation, artifacts va monitoring.
- Snowflake Streams/Tasks de so sanh voi dbt orchestration.
- Demo Zero-Copy Cloning, Time Travel, Fail-safe va Secure Data Sharing.
- Cortex AI enrichment co danh gia chat luong va chi phi.
- Bo test SQL va runbook van hanh.

1.1. Định nghĩa "hoàn thành"

Mot lan demo thanh cong phai cho thay:

1. Ba batch file duoc load.
2. Co duplicate, invalid record, late event va late customer change.
3. Chay lai batch khong tao duplicate nghiep vu.
4. SCD2 khong overlap va chi co mot current row.
5. Fact join dung dimension version tai business timestamp.
6. Mart doi soat duoc voi fact.
7. Task history, audit table va quarantine co bang chung.
8. Time Travel khoi phuc duoc mot thay doi sai.
9. Clone thay doi doc lap voi source.
10. AI enrichment khong goi lai model cho feedback khong thay doi.

2. Công nghệ và nền tảng

2.1. Thành phần bắt buộc

Thanh phan	Vai tro	Lua chon trong tutorial
Snowflake account	Database, compute, orchestration, sharing, AI	Trial hoac account cua cong ty
Snowsight	SQL worksheet, query profile, task graph	Trinh duyet web
dbt Core + dbt-snowflake	Transformation, test, lineage, deployment	Cong cu chinh
Amazon S3	Landing zone cua du lieu nguon	Bucket `landing1310`
AWS IAM	Xac thuc Snowflake vao S3 khong dung access key	IAM role + policy
Git	Quan ly version	Local repository
Python 3.11+	Sinh du lieu mau	Standard library la du
VS Code	Sua SQL/Markdown	Tuy chon

2.2. Thành phần tùy chọn

- Snowpipe auto-ingest: can cloud event notification.
- Snowflake CLI: chi can neu muon chay SQL/upload artifact tu terminal.
- BI tool: Power BI, Tableau, Sigma hoac Snowsight dashboard.
- CI/CD: GitHub Actions, GitLab CI hoac Azure DevOps.
- dbt: phu hop neu muon quan ly transformation theo model; tutorial goc van dung Snowflake SQL de thay ro co che.

2.3. Phạm vi edition và quyền

Truoc khi lam, xac nhan:

```
SELECT CURRENT_ACCOUNT(), CURRENT_REGION(), CURRENT_VERSION();
SELECT CURRENT_USER(), CURRENT_ROLE(), CURRENT_WAREHOUSE();
SHOW PARAMETERS LIKE 'DATA_RETENTION_TIME_IN_DAYS' IN ACCOUNT;
```

Mot so gioi han:

- Time Travel mo rong den 90 ngay can edition phu hop; Standard thuong co retention mac dinh 1 ngay.
- Secure Data Sharing end-to-end can provider va consumer account.
- Cortex AI phu thuoc region, model access va quyen.
- Resource Monitor khong bao phu moi serverless/AI spend; dung Budget/usage views cho cac dich vu do.

3. Cài đặt và setup

3.1. Tạo Snowflake account

Neu khong co account cong ty:

1. Tao trial account tu trang Snowflake.
2. Chon cloud provider va region co Cortex AI neu muon lam phan AI.
3. Luu lai:

- Account identifier.
- Username.
- Login URL.

4. Bat MFA neu account cho phep.
5. Khong commit password, private key hay token vao Git.

Sau khi dang nhap, vao **Projects > Worksheets > SQL Worksheet**, chon `ACCOUNTADMIN` va mot warehouse dang co, roi chay:

```
SELECT CURRENT_ACCOUNT(), CURRENT_REGION(), CURRENT_VERSION();
SELECT CURRENT_USER(), CURRENT_ROLE(), CURRENT_WAREHOUSE();
SHOW PARAMETERS LIKE 'DATA_RETENTION_TIME_IN_DAYS' IN ACCOUNT;
```

Day la noi "xac nhan account". Neu `CURRENT_WAREHOUSE()` la `NULL`, chon warehouse trong context bar cua worksheet.

3.2. Cài dbt Core cho Snowflake

Tutorial dung Python virtual environment de project doc lap:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install dbt-core dbt-snowflake
dbt --version
```

Windows PowerShell kich hoat environment bang:

```
py -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install dbt-core dbt-snowflake
```

Tao project o repository root:

```
dbt init abc_retail_dbt
```

Tao `~/.dbt/profiles.yml`. Vi du cho external browser:

```
abc_retail_dbt:
  target: dev
  outputs:
    dev:
      type: snowflake
      account: "ORGNAME-ACCOUNTNAME"
      user: "YOUR_USER"
      authenticator: externalbrowser
      role: ABC_ENGINEER
      database: ABC_RETAIL
      warehouse: ABC_TRANSFORM_WH
      schema: DBT_DEV
      threads: 4
      client_session_keep_alive: false
```

Neu dung password:

```
export DBT_SNOWFLAKE_PASSWORD='...'
```

va trong profile dung `password: "{{ env_var('DBT_SNOWFLAKE_PASSWORD') }}"`. Khong commit `profiles.yml`, password, private key hay token.

Chua chay `dbt debug` ngay: database, warehouse va role se duoc tao o Muc 4.

3.3. Cấu trúc repository

```
abc-retail-snowflake/
|-- README.md
|-- snowflake_end_to_end_assignment.md
|-- landing/
|   |-- clickstream_events/event_date=.../batch_id=.../
|   |-- orders/order_date=.../batch_id=.../
|   |-- customers/snapshot_date=.../batch_id=.../
|   |-- products/snapshot_date=.../batch_id=.../
|   |-- customer_feedback/feedback_date=.../batch_id=.../
|   `-- cdc_deletes/delete_date=.../batch_id=.../
|-- data/
|   `-- expected_results/customer_feedback_gold_labels.csv
|-- infra/
|   |-- 00_bootstrap.sql
|   |-- 01_s3_integration.sql
|   `-- 99_cleanup.sql
`-- abc_retail_dbt/
    |-- dbt_project.yml
    |-- packages.yml
    |-- macros/
    |   |-- create_platform_objects.sql
    |   `-- load_all_bronze.sql
    |-- models/
    |   |-- sources.yml
    |   |-- staging/
    |   |-- intermediate/
    |   |-- core/
    |   `-- marts/
    |-- snapshots/
    |-- tests/
    `-- seeds/
```

3.4. Cách chạy SQL

Trong tutorial nay, cac lenh co `USE ROLE ACCOUNTADMIN`, `CREATE INTEGRATION`, `CREATE SHARE` hoac recovery duoc chay trong **Snowsight Worksheet**. Nen dat Query Tag:

```
ALTER SESSION SET QUERY_TAG = 'abc_retail_tutorial:v2:bootstrap';
```

SQL transformation nam trong dbt model va chay bang:

```
cd abc_retail_dbt
dbt debug
dbt build
```

Snowflake CLI la tuy chon. Neu da cai, ban co the dung no de chay file `infra/*.sql`; neu khong, copy file vao Snowsight cho ket qua tuong duong.

4. Bootstrap: role, database, schema và warehouse

4.1. Role model

Su dung role chain:

```
ACCOUNTADMIN
    |
    v
ABC_ADMIN
    |
    +--> ABC_ENGINEER
    +--> ABC_ANALYST
    `--> ABC_SHARE_ADMIN
```

- `ABC_ADMIN`: so huu object cua project.
- `ABC_ENGINEER`: load, transform, task/procedure.
- `ABC_ANALYST`: read-only tren MART.
- `ABC_SHARE_ADMIN`: quan ly share, khong can quyen sua pipeline.

4.2. Script bootstrap

Chay block dau bang `ACCOUNTADMIN` trong account hoc tap. Trong production, nen de security admin va sysadmin tach nhiem vu.

```
USE ROLE ACCOUNTADMIN;

CREATE ROLE IF NOT EXISTS ABC_ADMIN;
CREATE ROLE IF NOT EXISTS ABC_ENGINEER;
CREATE ROLE IF NOT EXISTS ABC_ANALYST;
CREATE ROLE IF NOT EXISTS ABC_SHARE_ADMIN;

GRANT ROLE ABC_ENGINEER TO ROLE ABC_ADMIN;
GRANT ROLE ABC_ANALYST TO ROLE ABC_ADMIN;
GRANT ROLE ABC_SHARE_ADMIN TO ROLE ABC_ADMIN;

-- Thay YOUR_USER bang user dang hoc.
GRANT ROLE ABC_ADMIN TO USER YOUR_USER;

CREATE OR REPLACE WAREHOUSE ABC_INGEST_WH
  WAREHOUSE_SIZE = 'XSMALL'
  AUTO_SUSPEND = 60
  AUTO_RESUME = TRUE
  INITIALLY_SUSPENDED = TRUE;

CREATE OR REPLACE WAREHOUSE ABC_TRANSFORM_WH
  WAREHOUSE_SIZE = 'XSMALL'
  AUTO_SUSPEND = 60
  AUTO_RESUME = TRUE
  INITIALLY_SUSPENDED = TRUE;

CREATE OR REPLACE WAREHOUSE ABC_BI_WH
  WAREHOUSE_SIZE = 'XSMALL'
  AUTO_SUSPEND = 60
  AUTO_RESUME = TRUE
  INITIALLY_SUSPENDED = TRUE;

CREATE OR REPLACE DATABASE ABC_RETAIL
  DATA_RETENTION_TIME_IN_DAYS = 1
  COMMENT = 'ABC Retail learning platform';

CREATE SCHEMA ABC_RETAIL.LANDING;
CREATE SCHEMA ABC_RETAIL.BRONZE;
CREATE SCHEMA ABC_RETAIL.SILVER;
CREATE SCHEMA ABC_RETAIL.CORE_DWH;
CREATE SCHEMA ABC_RETAIL.MART;
CREATE SCHEMA ABC_RETAIL.GOVERNANCE;
CREATE SCHEMA ABC_RETAIL.SANDBOX;

GRANT USAGE ON WAREHOUSE ABC_INGEST_WH TO ROLE ABC_ENGINEER;
GRANT USAGE ON WAREHOUSE ABC_TRANSFORM_WH TO ROLE ABC_ENGINEER;
GRANT USAGE ON WAREHOUSE ABC_BI_WH TO ROLE ABC_ANALYST;

GRANT USAGE ON DATABASE ABC_RETAIL TO ROLE ABC_ENGINEER;
GRANT USAGE ON ALL SCHEMAS IN DATABASE ABC_RETAIL TO ROLE ABC_ENGINEER;
GRANT CREATE TABLE, CREATE VIEW, CREATE STAGE, CREATE FILE FORMAT,
      CREATE STREAM, CREATE TASK, CREATE PROCEDURE, CREATE DYNAMIC TABLE
  ON ALL SCHEMAS IN DATABASE ABC_RETAIL TO ROLE ABC_ENGINEER;

GRANT USAGE ON DATABASE ABC_RETAIL TO ROLE ABC_ANALYST;
GRANT USAGE ON SCHEMA ABC_RETAIL.MART TO ROLE ABC_ANALYST;
GRANT SELECT ON FUTURE TABLES IN SCHEMA ABC_RETAIL.MART TO ROLE ABC_ANALYST;
GRANT SELECT ON FUTURE VIEWS IN SCHEMA ABC_RETAIL.MART TO ROLE ABC_ANALYST;
```

Kiểm tra

```
SHOW ROLES LIKE 'ABC_%';
SHOW WAREHOUSES LIKE 'ABC_%';
SHOW SCHEMAS IN DATABASE ABC_RETAIL;
SHOW GRANTS TO ROLE ABC_ENGINEER;
```

4.3. Resource Monitor

Resource Monitor can quyen cao va chi kiem soat user-managed warehouse:

```
USE ROLE ACCOUNTADMIN;

CREATE OR REPLACE RESOURCE MONITOR ABC_MONTHLY_RM
  WITH CREDIT_QUOTA = 20
  FREQUENCY = MONTHLY
  START_TIMESTAMP = IMMEDIATELY
  TRIGGERS
    ON 75 PERCENT DO NOTIFY
    ON 90 PERCENT DO SUSPEND
    ON 100 PERCENT DO SUSPEND_IMMEDIATE;
```

```
ALTER WAREHOUSE ABC_INGEST_WH SET RESOURCE_MONITOR = ABC_MONTHLY_RM;
ALTER WAREHOUSE ABC_TRANSFORM_WH SET RESOURCE_MONITOR = ABC_MONTHLY_RM;
ALTER WAREHOUSE ABC_BI_WH SET RESOURCE_MONITOR = ABC_MONTHLY_RM;
```

Luu y: serverless Snowpipe, Automatic Clustering va Cortex AI khong duoc Resource Monitor kiem soat day du. Can theo doi usage views va Budget.

5. Tạo dữ liệu mẫu

5.1. Bộ dữ liệu landing thực tế

Repository đã có bộ dữ liệu xác định bởi `landing/_manifest.json`:

Nguồn	Records	Batches/files	Format
clickstream_events	500,000	345	JSON Lines
orders	80,160	345	JSON và Parquet
customers	30,000	854	CSV và JSON
dim_fx_rates	64	1	CSV
products	1,060, gồm 60 updates	2	CSV
customer_feedback	5,000	23	JSON Lines
cdc_deletes	300 tombstones	12	JSON Lines

Path có Hive-style partitions:

```
landing/<source>/
  load_date=YYYY-MM-DD/
    hour=HH/
      batch_id=<source>_YYYYMMDD_HH/
        <file>
```

Các source ban đầu chứa duplicate, late arrival, invalid timestamp, parse errors và stale updates. Dữ liệu bổ sung được tạo bằng `scripts/augment_landing_data.py`; nhãn kiểm thử AI nằm tại `data/expected_results/customer_feedback_gold_labels.csv`.

5.2. Ví dụ JSON

`landing/clickstream_events/.../events.json`:

```
{
  "event_id": "evt_0000063580",
  "event_time": "2026-05-07T20:06:53+01:00",
  "ingest_time": "2026-05-13T19:38:53Z",
  "user_id": "user_006872",
  "session_id": "sess_0000043449",
  "event_type": "view",
  "page": "/product/sku_00779",
  "device": "android|samsung_s24",
  "attributes": {"campaign": "summer", "sku": "sku_00779"}
}
{
  "event_id": "evt_0000076257",
  "event_time": "2026-05-07T01:19:48+07:00",
  "ingest_time": "2026-05-13T19:01:48Z",
  "user_id": "user_019769",
  "session_id": "sess_0000052087",
  "event_type": "view",
  "page": "/product/sku_00219",
  "device": {"os": "ios", "model": "iphone_15"},
  "attributes": {"campaign": "organic", "sku": "sku_00219"}
}
```

`landing/orders/.../orders.json`:

```
{
  "order_id": "order_00010657",
  "order_time": "2026-05-07T02:32:04+07:00",
  "user_id": "user_003190",
  "items": [{"sku": "sku_00247", "qty": 2, "unit_price": 221.73}],
  "currency": "VND",
  "discount_code": null,
  "shipping_address": {"country": "SG", "city": "Ritaberg"},
  "status": "created",
  "updated_at": "2026-05-06T19:32:04Z"
}
```

Các file giờ sau có thể chứa update cho cùng `order_id` với `updated_at` mới hơn, hoặc stale update đến sau nhưng có `updated_at` cũ hơn.

`landing/cdc_deletes/.../deletes.json` chứa source tombstone tách riêng:

```
{
  "entity_type": "order",
  "business_key": "order_00010657",
  "operation": "D",
  "changed_at": "2026-05-10T08:00:00Z",
  "reason": "privacy_request",
  "source_lsn": "order-delete-00000001"
}
```

5.3. Nguyên tắc sinh dữ liệu

- Du lieu mau phai nho de de debug.
- Moi edge case co expected result.
- Khong dung du lieu PII that.
- Co the sinh 100,000+ rows rieng cho bai performance.
- Timestamps nen co timezone ro rang.

6. Kết nối S3, stage và file format

Du lieu trong `landing/` da duoc upload len bucket `landing1310`. Vi vay, luong chinh cua tutorial doc truc tiep tu **S3 external stage**. Khong can upload lai landing data vao Snowflake internal stage.

6.1. Xác định đúng S3 prefix

Truoc khi tao integration, kiem tra object key:

```
aws s3 ls s3://landing1310/
aws s3 ls s3://landing1310/landing/
```

Tutorial mac dinh ban da upload nguyen folder `landing`, nen object co dang:

```
s3://landing1310/landing/clickstream_events/event_date=.../batch_id=.../events.json
s3://landing1310/landing/orders/order_date=.../batch_id=.../orders.parquet
s3://landing1310/landing/customers/snapshot_date=.../batch_id=.../customers.csv
s3://landing1310/landing/products/snapshot_date=.../batch_id=.../products.csv
s3://landing1310/landing/customer_feedback/feedback_date=.../batch_id=.../feedback.json
s3://landing1310/landing/cdc_deletes/delete_date=.../batch_id=.../deletes.json
s3://landing1310/landing/fx_rates/rate_date=.../fx_rates.csv
```

Neu lenh `aws s3 ls s3://landing1310/` hien truc tiep cac folder `clickstream_events/`, `orders/`, ... thi ban da upload **noi dung** cua `landing/` vao root. Khi do, thay moi URL `s3://landing1310/landing/` trong chuong nay bang `s3://landing1310/`.

Khong de `_manifest.json` nam trong pattern load business table. File nay chi dung de doi soat object, size va checksum.

6.2. Tạo IAM policy chỉ đọc cho landing

Trong AWS Console, vao **IAM > Policies > Create policy > JSON** va tao policy vi du `snowflake-abc-retail-landing-read`:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadLandingObjects",
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:GetObjectVersion"],
      "Resource": "arn:aws:s3:::landing1310/landing/*"
    },
    {
      "Sid": "ListLandingPrefix",
      "Effect": "Allow",
      "Action": ["s3:ListBucket", "s3:GetBucketLocation"],
      "Resource": "arn:aws:s3:::landing1310",
      "Condition": {
        "StringLike": {
          "s3:prefix": ["landing", "landing/*"]
        }
      }
    }
  ]
}
```

Neu landing nam o root bucket, doi object resource va dieu kien `s3:prefix` thanh wildcard bao phu toan bo object trong bucket. Policy tren chi cho doc. Neu muon unload vao S3, tao mot prefix `outbound/` va policy ghi rieng co `s3:PutObject`; khong mo quyen ghi cho landing neu khong can.

6.3. Tạo IAM role

Trong **IAM > Roles > Create role**:

1. Trusted entity type: **AWS account**.
2. Chon **Another AWS account** va tam thoi nhap AWS account ID cua ban.
3. Bat **Require external ID**, tam nhap `0000`.
4. Attach policy `snowflake-abc-retail-landing-read`.
5. Dat role name `snowflake-abc-retail-s3-role`.
6. Luu Role ARN, vi du

`arn:aws:iam::<AWS_ACCOUNT_ID>:role/snowflake-abc-retail-s3-role`.

Trust policy se duoc cap nhat bang Snowflake IAM user ARN va external ID that o buoc 6.5.

6.4. Tạo Storage Integration

`CREATE STORAGE INTEGRATION` can `ACCOUNTADMIN` hoac role co global privilege `CREATE INTEGRATION`:

```
USE ROLE ACCOUNTADMIN;

CREATE OR REPLACE STORAGE INTEGRATION ABC_RETAIL_S3_INT
  TYPE = EXTERNAL_STAGE
  STORAGE_PROVIDER = 'S3'
  ENABLED = TRUE
  STORAGE_AWS_ROLE_ARN =
    'arn:aws:iam::<AWS_ACCOUNT_ID>:role/snowflake-abc-retail-s3-role'
  STORAGE_ALLOWED_LOCATIONS = ('s3://landing1310/landing/');

GRANT USAGE ON INTEGRATION ABC_RETAIL_S3_INT TO ROLE ABC_ENGINEER;

DESC INTEGRATION ABC_RETAIL_S3_INT;
```

Neu landing nam o root, `STORAGE_ALLOWED_LOCATIONS` phai la `('s3://landing1310/')`. Ghi lai hai gia tri tu `DESC INTEGRATION`:

- `STORAGE_AWS_IAM_USER_ARN`
- `STORAGE_AWS_EXTERNAL_ID`

Khong chay lai `CREATE OR REPLACE STORAGE INTEGRATION` tuy y sau khi da cau hinh trust policy: integration moi co the sinh external ID moi.

6.5. Cập nhật trust policy và xác thực

Quay lai IAM role, chon **Trust relationships > Edit trust policy**:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "<STORAGE_AWS_IAM_USER_ARN>"
      },
      "Action": "sts:AssumeRole",
      "Condition": {
        "StringEquals": {
          "sts:ExternalId": "<STORAGE_AWS_EXTERNAL_ID>"
        }
      }
    }
  ]
}
```

Thay hai placeholder bang dung gia tri Snowflake tra ve. Sau khi luu, xac thuc:

```
USE ROLE ACCOUNTADMIN;

SELECT SYSTEM$VALIDATE_STORAGE_INTEGRATION(
  'ABC_RETAIL_S3_INT',
  's3://landing1310/landing/',
  'validate.txt',
  'list'
);
```

Neu prefix o root, doi URL trong lenh validate. Ket qua phai cho thay `success = true`; neu fail, kiem tra Role ARN, trust policy, external ID, bucket policy, KMS key policy neu object dung SSE-KMS, va region.

6.6. File formats và S3 external stage

```
USE ROLE ABC_ENGINEER;
USE WAREHOUSE ABC_INGEST_WH;
USE DATABASE ABC_RETAIL;
USE SCHEMA LANDING;

CREATE OR REPLACE FILE FORMAT FF_JSON
  TYPE = JSON
  STRIP_OUTER_ARRAY = FALSE
  IGNORE_UTF8_ERRORS = FALSE;

CREATE OR REPLACE FILE FORMAT FF_CSV
  TYPE = CSV
  FIELD_DELIMITER = ','
  SKIP_HEADER = 1
  FIELD_OPTIONALLY_ENCLOSED_BY = '"'
  EMPTY_FIELD_AS_NULL = TRUE
  NULL_IF = ('', 'NULL', 'null')
  ERROR_ON_COLUMN_COUNT_MISMATCH = FALSE;

CREATE OR REPLACE FILE FORMAT FF_PARQUET
  TYPE = PARQUET;

-- Đọc mỗi physical line như text để giữ được malformed JSON.
CREATE OR REPLACE FILE FORMAT FF_RAW_LINE
  TYPE = CSV
  FIELD_DELIMITER = NONE
  RECORD_DELIMITER = '\n'
  SKIP_HEADER = 0
  ESCAPE_UNENCLOSED_FIELD = NONE
  FIELD_OPTIONALLY_ENCLOSED_BY = NONE;

CREATE OR REPLACE STAGE STG_S3_LANDING
  URL = 's3://landing1310/landing/'
  STORAGE_INTEGRATION = ABC_RETAIL_S3_INT
  DIRECTORY = (
    ENABLE = TRUE
    REFRESH_ON_CREATE = TRUE
    AUTO_REFRESH = FALSE
  )
  COMMENT = 'Read-only landing data in S3 bucket landing1310';

-- Internal stage chỉ cho artifact local không nằm trong landing/.
CREATE OR REPLACE STAGE STG_TEST_ASSETS
  COMMENT = 'Expected labels and local test artifacts';

-- Internal stage cho ví dụ unload ở Chương 19.
CREATE OR REPLACE STAGE STG_OUTBOUND
  COMMENT = 'Unloaded marts';
```

Nếu landing nằm ở root bucket, đổi `URL` của stage thành `s3://landing1310/`. Không gán một file format duy nhất cho stage vì stage chứa JSON, CSV và Parquet; truyền `FORMAT_NAME` tại query/COPY.

Kiểm tra:

```
LIST @ABC_RETAIL.LANDING.STG_S3_LANDING;

ALTER STAGE ABC_RETAIL.LANDING.STG_S3_LANDING REFRESH;

SELECT RELATIVE_PATH, SIZE, LAST_MODIFIED
FROM DIRECTORY(@ABC_RETAIL.LANDING.STG_S3_LANDING)
ORDER BY RELATIVE_PATH
LIMIT 100;

SELECT
  METADATA$FILENAME,
  METADATA$FILE_ROW_NUMBER,
  METADATA$FILE_LAST_MODIFIED,
  $1
FROM @ABC_RETAIL.LANDING.STG_S3_LANDING/clickstream_events/
  (FILE_FORMAT => 'ABC_RETAIL.LANDING.FF_RAW_LINE')
LIMIT 10;
```

`LIST` đọc trực tiếp object list; directory table giúp query metadata bằng SQL. Khi `AUTO_REFRESH = FALSE`, chạy `ALTER STAGE ... REFRESH` sau khi có batch mới. Ở production, có thể bật auto-refresh bằng S3 event notification.

6.7. Tùy chọn Snowpipe auto-ingest

Batch demo trong tutorial vẫn dùng `COPY INTO` để dễ kiểm soát và debug. Nếu cần near-real-time, tạo một pipe riêng cho từng source/prefix. Ví dụ cho clickstream:

```
CREATE OR REPLACE PIPE ABC_RETAIL.BRONZE.PIPE_CLICKSTREAM
  AUTO_INGEST = TRUE
AS
COPY INTO ABC_RETAIL.BRONZE.BRONZE_CLICKSTREAM
  (RAW_TEXT, RAW_PAYLOAD, IS_PARSE_VALID,
   SOURCE_FILE, SOURCE_ROW_NUMBER, FILE_LAST_MODIFIED,
   BATCH_ID, INGEST_TS, PAYLOAD_HASH)
FROM (
  SELECT
    $1::VARCHAR,
    TRY_PARSE_JSON($1::VARCHAR),
    TRY_PARSE_JSON($1::VARCHAR) IS NOT NULL,
    METADATA$FILENAME,
    METADATA$FILE_ROW_NUMBER,
    METADATA$FILE_LAST_MODIFIED,
    REGEXP_SUBSTR(METADATA$FILENAME, 'batch_id=([^/]+)', 1, 1, 'e', 1),
    CURRENT_TIMESTAMP(),
    SHA2($1::VARCHAR, 256)
  FROM @ABC_RETAIL.LANDING.STG_S3_LANDING/clickstream_events/
    (FILE_FORMAT => 'ABC_RETAIL.LANDING.FF_RAW_LINE')
)
PATTERN = '.*events[.]json'
ON_ERROR = 'CONTINUE';

SHOW PIPES LIKE 'PIPE_CLICKSTREAM' IN SCHEMA ABC_RETAIL.BRONZE;
SELECT SYSTEM$PIPE_STATUS('ABC_RETAIL.BRONZE.PIPE_CLICKSTREAM');
```

Lay ARN trong cot `notification_channel` cua `SHOW PIPES`, sau do tai S3 bucket `landing1310` tao Event Notification:

- Event: `s3:ObjectCreated:*`.
- Destination: SQS queue ARN do Snowflake cung cap.
- Prefix: `landing/clickstream_events/` (bo `landing/` neu du lieu o root).
- Suffix: `.json`.

Moi source can pipe va filter khong overlap. Neu bucket da co notification trung prefix, dung SNS fan-out thay vi tao rule xung dot. Sau khi setup event, dung `ALTER PIPE ... REFRESH` theo huong dan Snowflake de nap backlog da ton tai truoc thoi diem bat notification.

6.8. Cấu hình dbt project

Trong `abc_retail_dbt/dbt_project.yml`:

```
name: abc_retail_dbt
version: 1.0.0
config-version: 2
profile: abc_retail_dbt

model-paths: ["models"]
snapshot-paths: ["snapshots"]
test-paths: ["tests"]
macro-paths: ["macros"]
seed-paths: ["seeds"]
clean-targets: ["target", "dbt_packages"]

models:
  abc_retail_dbt:
    staging:
      +schema: SILVER
      +materialized: view
      +tags: ["silver"]
    intermediate:
      +schema: SILVER
      +materialized: ephemeral
    core:
      +schema: CORE_DWH
      +materialized: incremental
      +tags: ["core"]
    marts:
      +schema: MART
      +materialized: table
      +tags: ["mart"]
```

Luu y: tuy `generate_schema_name`, dbt mac dinh co the tao schema ghep nhu `DBT_DEV_SILVER`. Cho project hoc tap muon dung dung `SILVER`, `CORE_DWH`, `MART`, tao macro:

```
-- macros/generate_schema_name.sql
{% macro generate_schema_name(custom_schema_name, node) -%}
  {%- if custom_schema_name is none -%}
```

```
    {{ target.schema }}
  {%- else -%}
    {{ custom_schema_name | trim }}
  {%- endif -%}
{%- endmacro %}
```

Chi dung cach nay khi moi developer co database/schema cach ly ro. Trong team, nen giu convention schema prefix cua dbt de tranh ghi de nhau.

6.9. Khai báo Bronze sources

`models/sources.yml`:

```
version: 2

sources:
  - name: bronze
    database: ABC_RETAIL
    schema: BRONZE
    freshness:
      warn_after: {count: 24, period: hour}
    loaded_at_field: ingest_ts
    tables:
      - name: bronze_clickstream
      - name: bronze_orders
      - name: bronze_customers
      - name: bronze_products
      - name: bronze_fx_rates
      - name: bronze_customer_feedback
      - name: bronze_cdc_deletes
```

Sau khi Muc 4 va Muc 6 hoan thanh:

```
cd abc_retail_dbt
dbt debug
dbt parse
```

`dbt debug` phai ket noi thanh cong. `dbt parse` phai khong co compilation error. Neu role `ABC_ENGINEER` chua co `USAGE` tren warehouse/database/schema, quay lai grants o Muc 4.

6.10. Ranh giới trách nhiệm

dbt khong tu dong thay the moi Snowflake object:

Object/cong viec	Cach quan ly trong project
Role, warehouse, database	`infra/00_bootstrap.sql`, chay Snowsight
Storage Integration, IAM	AWS Console + Snowsight
File format, stage, Bronze table	dbt macro `run-operation` hoac infra SQL
COPY INTO Bronze	dbt macro `load_all_bronze`
Silver/Core/Mart	dbt models
SCD1	Incremental model voi `merge`
SCD2 don gian	dbt snapshot
SCD2 late-arriving	Custom timeline model/procedure
Data quality	Generic tests + singular tests
Schedule	dbt Cloud job, Airflow, GitHub Actions; Task la bai so sanh
Clone, Time Travel, Share	Snowsight/infra SQL

7. Bronze layer: raw, append-only và lineage

7.1. Table design

Dung mot mau chung:

```
CREATE OR REPLACE TABLE ABC_RETAIL.BRONZE.BRONZE_CLICKSTREAM (
  RAW_TEXT VARCHAR,
  RAW_PAYLOAD VARIANT,
  IS_PARSE_VALID BOOLEAN NOT NULL DEFAULT TRUE,
  SOURCE_FILE VARCHAR NOT NULL,
  SOURCE_ROW_NUMBER NUMBER,
  FILE_LAST_MODIFIED TIMESTAMP_TZ,
  BATCH_ID VARCHAR NOT NULL,
  INGEST_TS TIMESTAMP_TZ NOT NULL DEFAULT CURRENT_TIMESTAMP(),
  PAYLOAD_HASH VARCHAR NOT NULL,
  CONSTRAINT UQ_BRONZE_CLICKSTREAM UNIQUE
    (SOURCE_FILE, SOURCE_ROW_NUMBER, PAYLOAD_HASH)
);
```

Snowflake standard table constraints nhu UNIQUE thuong mang tinh metadata, khong nen dua vao no de enforcement. Idempotency phai do load pattern va test.

Cac Bronze table khac dung cung metadata:

```
CREATE OR REPLACE TABLE ABC_RETAIL.BRONZE.BRONZE_ORDERS
  LIKE ABC_RETAIL.BRONZE.BRONZE_CLICKSTREAM;
CREATE OR REPLACE TABLE ABC_RETAIL.BRONZE.BRONZE_CUSTOMERS
  LIKE ABC_RETAIL.BRONZE.BRONZE_CLICKSTREAM;
CREATE OR REPLACE TABLE ABC_RETAIL.BRONZE.BRONZE_PRODUCTS
  LIKE ABC_RETAIL.BRONZE.BRONZE_CLICKSTREAM;
CREATE OR REPLACE TABLE ABC_RETAIL.BRONZE.BRONZE_FX_RATES
  LIKE ABC_RETAIL.BRONZE.BRONZE_CLICKSTREAM;
CREATE OR REPLACE TABLE ABC_RETAIL.BRONZE.BRONZE_CUSTOMER_FEEDBACK
  LIKE ABC_RETAIL.BRONZE.BRONZE_CLICKSTREAM;
CREATE OR REPLACE TABLE ABC_RETAIL.BRONZE.BRONZE_CDC_DELETES
  LIKE ABC_RETAIL.BRONZE.BRONZE_CLICKSTREAM;
```

Dat DDL tren trong macro `macros/create_platform_objects.sql`:

```
{% macro create_platform_objects() %}
  {% set ddl %}
    CREATE TABLE IF NOT EXISTS ABC_RETAIL.BRONZE.BRONZE_CLICKSTREAM (
      RAW_TEXT VARCHAR,
      RAW_PAYLOAD VARIANT,
      IS_PARSE_VALID BOOLEAN NOT NULL DEFAULT TRUE,
      SOURCE_FILE VARCHAR NOT NULL,
      SOURCE_ROW_NUMBER NUMBER,
      FILE_LAST_MODIFIED TIMESTAMP_TZ,
      BATCH_ID VARCHAR NOT NULL,
      INGEST_TS TIMESTAMP_TZ NOT NULL DEFAULT CURRENT_TIMESTAMP(),
      PAYLOAD_HASH VARCHAR NOT NULL
    )
  {% endset %}
  {% do run_query(ddl) %}
  {{ log("Bronze platform objects are ready", info=true) }}
{% endmacro %}
```

Chay macro mot lan:

```
dbt run-operation create_platform_objects
```

Trong implementation thuc te, macro phai tao du bay table Bronze, quarantine va audit. DDL trong cac muc 7-8 la noi dung dua vao macro nay.

7.2. Macro ingestion và cách chạy

`COPY INTO` la statement van hanh, khong phai SELECT model. Dat no trong `macros/load_all_bronze.sql`, khong dat trong `models/`:

```
{% macro load_all_bronze(batch_id='manual') %}
  {{ log("Loading Bronze, run batch: " ~ batch_id, info=true) }}

  {% set copy_clickstream %}
    COPY INTO ABC_RETAIL.BRONZE.BRONZE_CLICKSTREAM
      (RAW_TEXT, RAW_PAYLOAD, IS_PARSE_VALID,
       SOURCE_FILE, SOURCE_ROW_NUMBER, FILE_LAST_MODIFIED,
       BATCH_ID, INGEST_TS, PAYLOAD_HASH)
    FROM (
      SELECT
        $1::VARCHAR,
        TRY_PARSE_JSON($1::VARCHAR),
        TRY_PARSE_JSON($1::VARCHAR) IS NOT NULL,
```

```
        METADATA$FILENAME,
        METADATA$FILE_ROW_NUMBER,
        METADATA$FILE_LAST_MODIFIED,
        REGEXP_SUBSTR(METADATA$FILENAME,
                      'batch_id=([^/]+)', 1, 1, 'e', 1),
        CURRENT_TIMESTAMP(),
        SHA2($1::VARCHAR, 256)
      FROM @ABC_RETAIL.LANDING.STG_S3_LANDING/clickstream_events/
        (FILE_FORMAT => 'ABC_RETAIL.LANDING.FF_RAW_LINE')
    )
    PATTERN = '.*events[.]json'
    ON_ERROR = 'CONTINUE'
  {% endset %}

  {% do run_query(copy_clickstream) %}
  {{ log("Clickstream Bronze loaded", info=true) }}
{% endmacro %}
```

Mo rong macro bang cac statement trong 7.3-7.6 cho orders, customers, products, FX, feedback va tombstones. Chay:

```
dbt run-operation load_all_bronze --args '{batch_id: manual_001}'
dbt source freshness
```

Khong dung `dbt run` de thuc thi `COPY INTO`: `dbt run` chi build models.

7.3. Load JSON bằng raw-line ingestion

Không dùng JSON file format cho bước đầu tiên, vì landing cố ý có malformed JSON. Đọc từng dòng thành `RAW_TEXT`, sau đó `TRY_PARSE_JSON`. Như vậy parse errors vẫn có raw payload để quarantine.

```
COPY INTO ABC_RETAIL.BRONZE.BRONZE_CLICKSTREAM
  (RAW_TEXT, RAW_PAYLOAD, IS_PARSE_VALID,
   SOURCE_FILE, SOURCE_ROW_NUMBER,
   FILE_LAST_MODIFIED, BATCH_ID, INGEST_TS, PAYLOAD_HASH)
FROM (
  SELECT
    $1::VARCHAR,
    TRY_PARSE_JSON($1::VARCHAR),
    TRY_PARSE_JSON($1::VARCHAR) IS NOT NULL,
    METADATA$FILENAME,
    METADATA$FILE_ROW_NUMBER,
    METADATA$FILE_LAST_MODIFIED,
    REGEXP_SUBSTR(METADATA$FILENAME, 'batch_id=([^/]+)', 1, 1, 'e', 1),
    CURRENT_TIMESTAMP(),
    SHA2($1::VARCHAR, 256)
  FROM @ABC_RETAIL.LANDING.STG_S3_LANDING/clickstream_events/
    (FILE_FORMAT => 'ABC_RETAIL.LANDING.FF_RAW_LINE')
)
PATTERN = '.*events[.]json'
ON_ERROR = 'CONTINUE';
```

Snowflake luu load history theo table/file. Neu load lai dung file name ma khong `FORCE=TRUE`, COPY bo qua file da load. Khong dung `FORCE=TRUE` trong normal run. Neu can replay, load vao temporary table roi `MERGE` theo `SOURCE_FILE + SOURCE_ROW_NUMBER + PAYLOAD_HASH`.

7.4. Quarantine parse errors

```
INSERT INTO ABC_RETAIL.GOVERNANCE.QUARANTINE_RECORDS (
  SOURCE_NAME, SOURCE_FILE, SOURCE_ROW_NUMBER, BATCH_ID,
  RAW_TEXT, RAW_PAYLOAD, ERROR_CODE, ERROR_MESSAGE, RUN_ID
)
SELECT
  'clickstream', SOURCE_FILE, SOURCE_ROW_NUMBER, BATCH_ID,
  RAW_TEXT, RAW_PAYLOAD, 'PARSE_ERROR',
  'TRY_PARSE_JSON returned NULL', :RUN_ID
FROM ABC_RETAIL.BRONZE.BRONZE_CLICKSTREAM
WHERE NOT IS_PARSE_VALID;
```

Sau load:

```
SELECT *
FROM TABLE(
  ABC_RETAIL.INFORMATION_SCHEMA.COPY_HISTORY(
    TABLE_NAME => 'ABC_RETAIL.BRONZE.BRONZE_ORDERS',
    START_TIME => DATEADD('hour', -24, CURRENT_TIMESTAMP())
  )
)
ORDER BY LAST_LOAD_TIME DESC;
```

7.5. CSV vào raw VARIANT

Vi CSV khong tu nhien thanh object, dung `OBJECT_CONSTRUCT_KEEP_NULL`:

```
COPY INTO ABC_RETAIL.BRONZE.BRONZE_CUSTOMERS
  (RAW_PAYLOAD, SOURCE_FILE, SOURCE_ROW_NUMBER,
   FILE_LAST_MODIFIED, BATCH_ID, INGEST_TS, PAYLOAD_HASH)
FROM (
  SELECT
    OBJECT_CONSTRUCT_KEEP_NULL(
      'user_id', $1,
      'email', $2,
      'phone', $3,
      'country', $4,
      'created_at', $5,
      'updated_at', $6
    ),
    METADATA$FILENAME,
    METADATA$FILE_ROW_NUMBER,
    METADATA$FILE_LAST_MODIFIED,
    REGEXP_SUBSTR(METADATA$FILENAME, 'batch_id=([^/]+)', 1, 1, 'e', 1),
    CURRENT_TIMESTAMP(),
    SHA2(CONCAT_WS('|', $1, $2, $3, $4, $5, $6), 256)
  FROM @ABC_RETAIL.LANDING.STG_S3_LANDING/customers/
    (FILE_FORMAT => 'ABC_RETAIL.LANDING.FF_CSV')
)
PATTERN = '.*customers[.]csv'
ON_ERROR = 'CONTINUE';
```

Customer JSON dùng raw-line pattern tương tự clickstream. Orders JSON dùng raw-line; orders Parquet dùng `FF_PARQUET` và transform `$1` thành cùng `RAW_PAYLOAD`. Hai `COPY` cùng đổ về `BRONZE_ORDERS`, nhưng phải dùng `PATTERN` riêng cho `.json` và `.parquet`.

7.6. Load Products, Feedback và CDC tombstones

Product CSV:

```
COPY INTO ABC_RETAIL.BRONZE.BRONZE_PRODUCTS (
  RAW_PAYLOAD, SOURCE_FILE, SOURCE_ROW_NUMBER, FILE_LAST_MODIFIED,
  BATCH_ID, INGEST_TS, PAYLOAD_HASH
)
FROM (
  SELECT
    OBJECT_CONSTRUCT_KEEP_NULL(
      'sku', $1,
      'product_name', $2,
      'brand', $3,
      'subcategory_code', $4,
      'subcategory_name', $5,
      'category_code', $6,
      'category_name', $7,
      'unit_cost', $8,
      'updated_at', $9
    ),
    METADATA$FILENAME,
    METADATA$FILE_ROW_NUMBER,
    METADATA$FILE_LAST_MODIFIED,
    REGEXP_SUBSTR(METADATA$FILENAME, 'batch_id=([^/]+)', 1, 1, 'e', 1),
    CURRENT_TIMESTAMP(),
    SHA2(CONCAT_WS('|', $1, $2, $3, $4, $5, $6, $7, $8, $9), 256)
  FROM @ABC_RETAIL.LANDING.STG_S3_LANDING/products/
    (FILE_FORMAT => 'ABC_RETAIL.LANDING.FF_CSV')
)
PATTERN = '.*products[.]csv';
```

Feedback và CDC tombstones dùng cùng raw-line pattern. Chỉ thay prefix, target table và file pattern:

```
-- Feedback:
-- prefix: customer_feedback/
-- target: BRONZE_CUSTOMER_FEEDBACK
-- pattern: .*feedback[.]json

-- CDC deletes:
-- prefix: cdc_deletes/
-- target: BRONZE_CDC_DELETES
-- pattern: .*deletes[.]json
```

Nên đóng gói raw-line `COPY` thành script/template dùng lại, nhưng vẫn giữ target table riêng để lineage, grants và stream lifecycle độc lập.

8. Quarantine, audit và data quality

8.1. Quarantine table

```
CREATE OR REPLACE TABLE ABC_RETAIL.GOVERNANCE.QUARANTINE_RECORDS (
  QUARANTINE_ID VARCHAR DEFAULT UUID_STRING(),
  SOURCE_NAME VARCHAR NOT NULL,
  SOURCE_FILE VARCHAR,
  SOURCE_ROW_NUMBER NUMBER,
  BATCH_ID VARCHAR,
  RAW_TEXT VARCHAR,
  RAW_PAYLOAD VARIANT,
  ERROR_CODE VARCHAR NOT NULL,
  ERROR_MESSAGE VARCHAR,
  QUARANTINED_AT TIMESTAMP_TZ DEFAULT CURRENT_TIMESTAMP(),
  RUN_ID VARCHAR
);
```

Mot record co the co nhieu loi. Neu can reconciliation theo record, them `IS_PRIMARY_ERROR` va chi dem primary error.

8.2. Audit table

```
CREATE OR REPLACE TABLE ABC_RETAIL.GOVERNANCE.AUDIT_PIPELINE_RUNS (
  RUN_ID VARCHAR,
  PIPELINE_NAME VARCHAR,
  TASK_NAME VARCHAR,
  BATCH_ID VARCHAR,
  STARTED_AT TIMESTAMP_TZ,
  ENDED_AT TIMESTAMP_TZ,
  STATUS VARCHAR,
  ROWS_READ NUMBER DEFAULT 0,
  ROWS_INSERTED NUMBER DEFAULT 0,
  ROWS_UPDATED NUMBER DEFAULT 0,
  ROWS_DELETED NUMBER DEFAULT 0,
  ROWS_QUARANTINED NUMBER DEFAULT 0,
  SOURCE_FILES ARRAY,
  QUERY_ID VARCHAR,
  ERROR_MESSAGE VARCHAR,
  PRIMARY KEY (RUN_ID, TASK_NAME)
);
```

8.3. Rule catalog

```
CREATE OR REPLACE TABLE ABC_RETAIL.GOVERNANCE.DQ_RULES (
  RULE_CODE VARCHAR,
  ENTITY_NAME VARCHAR,
  RULE_DESCRIPTION VARCHAR,
  SEVERITY VARCHAR,
  IS_ACTIVE BOOLEAN,
  PRIMARY KEY (RULE_CODE)
);

INSERT INTO ABC_RETAIL.GOVERNANCE.DQ_RULES VALUES
  ('MISSING_KEY', 'ALL', 'Business key is null', 'ERROR', TRUE),
  ('INVALID_TIMESTAMP', 'ALL', 'Timestamp cannot be parsed', 'ERROR', TRUE),
  ('INVALID_QUANTITY', 'ORDER_ITEM', 'Quantity must be positive', 'ERROR', TRUE),
  ('INVALID_EMAIL', 'CUSTOMER', 'Email format is invalid', 'WARN', TRUE),
  ('FX_RATE_MISSING', 'ORDER', 'No FX rate for order date', 'ERROR', TRUE),
  ('SCD_OVERLAP', 'CUSTOMER', 'SCD validity ranges overlap', 'ERROR', TRUE);
```

9. Silver layer: parse, type, deduplicate

Trong dbt, moi Silver dataset la mot model `SELECT`. dbt tao table/view theo `config`; khong copy `CREATE TABLE` vao model. Cac DDL ben duoi mo ta contract cot, con business query duoc dat trong file `.sql`.

Vi du `models/staging/stg_events.sql`:

```
{{
  config(
    materialized='incremental',
    incremental_strategy='merge',
    unique_key='event_id',
    on_schema_change='fail',
    tags=['silver']
```

```
  )
}}

with source_rows as (
  select *
  from {{ source('bronze', 'bronze_clickstream') }}
  where is_parse_valid
  {% if is_incremental() %}
    and ingest_ts >= (
      select dateadd(day, -3, coalesce(max(ingest_ts), '1900-01-01'))
      from {{ this }}
    )
  {% endif %}
),
parsed as (
  select
    raw_payload:event_id::varchar as event_id,
    try_to_timestamp_tz(raw_payload:event_time::varchar) as event_time_utc,
    raw_payload:user_id::varchar as user_id,
    raw_payload:session_id::varchar as session_id,
    raw_payload:event_type::varchar as event_type,
    source_file,
    batch_id,
    ingest_ts,
    payload_hash as record_hash
  from source_rows
),
deduplicated as (
  select *
  from parsed
  qualify row_number() over (
    partition by event_id
    order by ingest_ts desc, source_file desc
  ) = 1
)
select * from deduplicated
```

Lookback ba ngay la guardrail cho late arrival, khong phai dap an chung. Dat window theo SLA va kiem thu voi du lieu late thuc te.

9.1. Silver Events

```
CREATE OR REPLACE TABLE ABC_RETAIL.SILVER.SILVER_EVENTS (
  EVENT_ID VARCHAR,
  EVENT_TIME_UTC TIMESTAMP_TZ,
  INGEST_TIME_UTC TIMESTAMP_TZ,
  USER_ID VARCHAR,
  SESSION_ID VARCHAR,
  ANALYTIC_USER_ID VARCHAR,
  EVENT_TYPE VARCHAR,
  PAGE VARCHAR,
  DEVICE_OS VARCHAR,
  DEVICE_MODEL VARCHAR,
  ATTRIBUTES VARIANT,
  SOURCE_FILE VARCHAR,
  BATCH_ID VARCHAR,
  INGEST_TS TIMESTAMP_TZ,
  RECORD_HASH VARCHAR
);
```

Parse `device` ca object va string:

```
WITH parsed AS (
  SELECT
    RAW_PAYLOAD:event_id::VARCHAR AS EVENT_ID,
    TRY_TO_TIMESTAMP_TZ(RAW_PAYLOAD:event_time::VARCHAR) AS EVENT_TIME_UTC,
    TRY_TO_TIMESTAMP_TZ(RAW_PAYLOAD:ingest_time::VARCHAR) AS INGEST_TIME_UTC,
    RAW_PAYLOAD:user_id::VARCHAR AS USER_ID,
    RAW_PAYLOAD:session_id::VARCHAR AS SESSION_ID,
    RAW_PAYLOAD:event_type::VARCHAR AS EVENT_TYPE,
    RAW_PAYLOAD:page::VARCHAR AS PAGE,
    CASE
      WHEN TYPEOF(RAW_PAYLOAD:device) = 'OBJECT'
        THEN RAW_PAYLOAD:device:os::VARCHAR
      ELSE SPLIT_PART(RAW_PAYLOAD:device::VARCHAR, '|', 1)
    END AS DEVICE_OS,
    CASE
      WHEN TYPEOF(RAW_PAYLOAD:device) = 'OBJECT'
        THEN RAW_PAYLOAD:device:model::VARCHAR
      ELSE SPLIT_PART(RAW_PAYLOAD:device::VARCHAR, '|', 2)
    END AS DEVICE_MODEL,
    RAW_PAYLOAD:attributes AS ATTRIBUTES,
    SOURCE_FILE, BATCH_ID, INGEST_TS,
    PAYLOAD_HASH
  FROM ABC_RETAIL.BRONZE.BRONZE_CLICKSTREAM
```

```
),
valid AS (
  SELECT *,
    COALESCE(USER_ID, 'anonymous:' || SESSION_ID) AS ANALYTIC_USER_ID
  FROM parsed
  WHERE EVENT_ID IS NOT NULL
    AND EVENT_TIME_UTC IS NOT NULL
    AND EVENT_TYPE IN ('view', 'add_to_cart', 'checkout', 'purchase')
)
MERGE INTO ABC_RETAIL.SILVER.SILVER_EVENTS t
USING (
  SELECT *
  FROM valid
  QUALIFY ROW_NUMBER() OVER (
    PARTITION BY EVENT_ID
    ORDER BY COALESCE(INGEST_TIME_UTC, INGEST_TS) DESC,
             INGEST_TS DESC, SOURCE_FILE DESC
  ) = 1
) s
ON t.EVENT_ID = s.EVENT_ID
WHEN MATCHED
  AND COALESCE(s.INGEST_TIME_UTC, s.INGEST_TS)
      > COALESCE(t.INGEST_TIME_UTC, t.INGEST_TS)
THEN UPDATE SET
  EVENT_TIME_UTC = s.EVENT_TIME_UTC,
  INGEST_TIME_UTC = s.INGEST_TIME_UTC,
  USER_ID = s.USER_ID,
  SESSION_ID = s.SESSION_ID,
  ANALYTIC_USER_ID = s.ANALYTIC_USER_ID,
  EVENT_TYPE = s.EVENT_TYPE,
  PAGE = s.PAGE,
  DEVICE_OS = s.DEVICE_OS,
  DEVICE_MODEL = s.DEVICE_MODEL,
  ATTRIBUTES = s.ATTRIBUTES,
  SOURCE_FILE = s.SOURCE_FILE,
  BATCH_ID = s.BATCH_ID,
  INGEST_TS = s.INGEST_TS,
  RECORD_HASH = s.PAYLOAD_HASH
WHEN NOT MATCHED THEN INSERT (
  EVENT_ID, EVENT_TIME_UTC, INGEST_TIME_UTC, USER_ID, SESSION_ID,
  ANALYTIC_USER_ID, EVENT_TYPE, PAGE, DEVICE_OS, DEVICE_MODEL,
  ATTRIBUTES, SOURCE_FILE, BATCH_ID, INGEST_TS, RECORD_HASH
) VALUES (
  s.EVENT_ID, s.EVENT_TIME_UTC, s.INGEST_TIME_UTC, s.USER_ID, s.SESSION_ID,
  s.ANALYTIC_USER_ID, s.EVENT_TYPE, s.PAGE, s.DEVICE_OS, s.DEVICE_MODEL,
  s.ATTRIBUTES, s.SOURCE_FILE, s.BATCH_ID, s.INGEST_TS, s.PAYLOAD_HASH
);
```

Ghi invalid events truoc khi merge:

```
INSERT INTO ABC_RETAIL.GOVERNANCE.QUARANTINE_RECORDS
  (SOURCE_NAME, SOURCE_FILE, SOURCE_ROW_NUMBER, BATCH_ID, RAW_PAYLOAD,
   ERROR_CODE, ERROR_MESSAGE, RUN_ID)
SELECT
  'clickstream', SOURCE_FILE, SOURCE_ROW_NUMBER, BATCH_ID, RAW_PAYLOAD,
  CASE
    WHEN RAW_PAYLOAD:event_id IS NULL THEN 'MISSING_KEY'
    WHEN TRY_TO_TIMESTAMP_TZ(RAW_PAYLOAD:event_time::VARCHAR) IS NULL
      THEN 'INVALID_TIMESTAMP'
    ELSE 'INVALID_EVENT_TYPE'
  END,
  'Event failed Silver validation',
  :RUN_ID
FROM ABC_RETAIL.BRONZE.BRONZE_CLICKSTREAM
WHERE RAW_PAYLOAD:event_id IS NULL
   OR TRY_TO_TIMESTAMP_TZ(RAW_PAYLOAD:event_time::VARCHAR) IS NULL
   OR RAW_PAYLOAD:event_type::VARCHAR NOT IN
      ('view', 'add_to_cart', 'checkout', 'purchase');
```

9.2. Silver Orders

Tach ba muc dich:

- `SILVER_ORDERS_CURRENT`: latest state, SCD1.
- `SILVER_ORDER_STATUS_HISTORY`: event history.
- `SILVER_ORDER_ITEMS`: line items cua latest order version.

```
CREATE OR REPLACE TABLE ABC_RETAIL.SILVER.SILVER_ORDERS_CURRENT (
  ORDER_ID VARCHAR,
  ORDER_TIME_UTC TIMESTAMP_TZ,
  USER_ID VARCHAR,
  CURRENCY VARCHAR,
  STATUS VARCHAR,
  COUNTRY_CODE VARCHAR,
```

```
  UPDATED_AT TIMESTAMP_TZ,
  SOURCE_FILE VARCHAR,
  BATCH_ID VARCHAR,
  RECORD_HASH VARCHAR,
  PRIMARY KEY (ORDER_ID)
);

CREATE OR REPLACE TABLE ABC_RETAIL.SILVER.SILVER_ORDER_STATUS_HISTORY (
  ORDER_ID VARCHAR,
  STATUS VARCHAR,
  STATUS_AT TIMESTAMP_TZ,
  BATCH_ID VARCHAR,
  RECORD_HASH VARCHAR
);

CREATE OR REPLACE TABLE ABC_RETAIL.SILVER.SILVER_ORDER_ITEMS (
  ORDER_ID VARCHAR,
  LINE_NUMBER NUMBER,
  SKU VARCHAR,
  QTY NUMBER(18,3),
  UNIT_PRICE_ORIGINAL NUMBER(18,4),
  CURRENCY VARCHAR,
  FX_RATE_TO_USD NUMBER(18,8),
  LINE_AMOUNT_USD NUMBER(18,4),
  UPDATED_AT TIMESTAMP_TZ,
  PRIMARY KEY (ORDER_ID, LINE_NUMBER)
);
```

9.3. Silver Customer Changes

```
CREATE OR REPLACE TABLE ABC_RETAIL.SILVER.SILVER_CUSTOMER_CHANGES (
  USER_ID VARCHAR,
  EMAIL VARCHAR,
  PHONE VARCHAR,
  COUNTRY_CODE VARCHAR,
  CREATED_AT TIMESTAMP_TZ,
  UPDATED_AT TIMESTAMP_TZ,
  RECORD_HASH VARCHAR,
  SOURCE_FILE VARCHAR,
  BATCH_ID VARCHAR,
  INGEST_TS TIMESTAMP_TZ
);
```

Landing dùng field `country`, không phải `country_code`. Chuẩn hóa ngay khi đưa vào Silver:

```
UPPER(TRIM(RAW_PAYLOAD:country::VARCHAR)) AS COUNTRY_CODE
```

Hash chỉ gom tracked attributes:

```
SHA2(
  CONCAT_WS('|',
    COALESCE(LOWER(TRIM(EMAIL)), '<NULL>'),
    COALESCE(REGEXP_REPLACE(PHONE, '[^0-9+]', ''), '<NULL>'),
    COALESCE(UPPER(TRIM(COUNTRY_CODE)), '<NULL>')
  ),
  256
) AS RECORD_HASH
```

9.4. FX rates và Products

FX file thực tế có schema `rate_date,currency,rate_to_usd`. Chuẩn hóa thành schema dùng chung:

```
CREATE OR REPLACE TABLE ABC_RETAIL.SILVER.SILVER_FX_RATES (
  RATE_DATE DATE,
  SOURCE_CURRENCY VARCHAR,
  TARGET_CURRENCY VARCHAR,
  RATE NUMBER(18,8),
  PRIMARY KEY (RATE_DATE, SOURCE_CURRENCY, TARGET_CURRENCY)
);

INSERT INTO ABC_RETAIL.SILVER.SILVER_FX_RATES
SELECT
  TRY_TO_DATE(RAW_PAYLOAD:rate_date::VARCHAR),
  UPPER(RAW_PAYLOAD:currency::VARCHAR),
  'USD',
  TRY_TO_DECIMAL(RAW_PAYLOAD:rate_to_usd::VARCHAR, 18, 8)
FROM ABC_RETAIL.BRONZE.BRONZE_FX_RATES
WHERE RAW_PAYLOAD IS NOT NULL;
```

Product master có 1.000 SKU và 60 update rows. Dùng SCD1 cho current product trước khi xây hai dimensional variants:

```
CREATE OR REPLACE TABLE ABC_RETAIL.SILVER.SILVER_PRODUCTS (
  SKU VARCHAR,
```

```
  PRODUCT_NAME VARCHAR,
  BRAND VARCHAR,
  SUBCATEGORY_CODE VARCHAR,
  SUBCATEGORY_NAME VARCHAR,
  CATEGORY_CODE VARCHAR,
  CATEGORY_NAME VARCHAR,
  UNIT_COST NUMBER(18,4),
  UPDATED_AT TIMESTAMP_TZ,
  RECORD_HASH VARCHAR,
  PRIMARY KEY (SKU)
);

MERGE INTO ABC_RETAIL.SILVER.SILVER_PRODUCTS t
USING (
  SELECT
    RAW_PAYLOAD:sku::VARCHAR AS SKU,
    RAW_PAYLOAD:product_name::VARCHAR AS PRODUCT_NAME,
    RAW_PAYLOAD:brand::VARCHAR AS BRAND,
    RAW_PAYLOAD:subcategory_code::VARCHAR AS SUBCATEGORY_CODE,
    RAW_PAYLOAD:subcategory_name::VARCHAR AS SUBCATEGORY_NAME,
    RAW_PAYLOAD:category_code::VARCHAR AS CATEGORY_CODE,
    RAW_PAYLOAD:category_name::VARCHAR AS CATEGORY_NAME,
    TRY_TO_DECIMAL(RAW_PAYLOAD:unit_cost::VARCHAR, 18, 4) AS UNIT_COST,
    TRY_TO_TIMESTAMP_TZ(RAW_PAYLOAD:updated_at::VARCHAR) AS UPDATED_AT,
    PAYLOAD_HASH AS RECORD_HASH
  FROM ABC_RETAIL.BRONZE.BRONZE_PRODUCTS
  QUALIFY ROW_NUMBER() OVER (
    PARTITION BY RAW_PAYLOAD:sku::VARCHAR
    ORDER BY TRY_TO_TIMESTAMP_TZ(RAW_PAYLOAD:updated_at::VARCHAR) DESC
  ) = 1
) s
ON t.SKU = s.SKU
WHEN MATCHED AND s.UPDATED_AT > t.UPDATED_AT THEN UPDATE SET
  PRODUCT_NAME = s.PRODUCT_NAME,
  BRAND = s.BRAND,
  SUBCATEGORY_CODE = s.SUBCATEGORY_CODE,
  SUBCATEGORY_NAME = s.SUBCATEGORY_NAME,
  CATEGORY_CODE = s.CATEGORY_CODE,
  CATEGORY_NAME = s.CATEGORY_NAME,
  UNIT_COST = s.UNIT_COST,
  UPDATED_AT = s.UPDATED_AT,
  RECORD_HASH = s.RECORD_HASH
WHEN NOT MATCHED THEN INSERT (
  SKU, PRODUCT_NAME, BRAND, SUBCATEGORY_CODE, SUBCATEGORY_NAME,
  CATEGORY_CODE, CATEGORY_NAME, UNIT_COST, UPDATED_AT, RECORD_HASH
) VALUES (
  s.SKU, s.PRODUCT_NAME, s.BRAND, s.SUBCATEGORY_CODE, s.SUBCATEGORY_NAME,
  s.CATEGORY_CODE, s.CATEGORY_NAME, s.UNIT_COST, s.UPDATED_AT, s.RECORD_HASH
);
```

9.5. Customer Feedback

```
CREATE OR REPLACE TABLE ABC_RETAIL.SILVER.SILVER_CUSTOMER_FEEDBACK (
  FEEDBACK_ID VARCHAR,
  USER_ID VARCHAR,
  ORDER_ID VARCHAR,
  FEEDBACK_TEXT VARCHAR,
  CREATED_AT TIMESTAMP_TZ,
  UPDATED_AT TIMESTAMP_TZ,
  LANGUAGE VARCHAR,
  FEEDBACK_HASH VARCHAR,
  SOURCE_FILE VARCHAR,
  BATCH_ID VARCHAR,
  PRIMARY KEY (FEEDBACK_ID)
);

MERGE INTO ABC_RETAIL.SILVER.SILVER_CUSTOMER_FEEDBACK t
USING (
  SELECT
    RAW_PAYLOAD:feedback_id::VARCHAR AS FEEDBACK_ID,
    RAW_PAYLOAD:user_id::VARCHAR AS USER_ID,
    RAW_PAYLOAD:order_id::VARCHAR AS ORDER_ID,
    RAW_PAYLOAD:feedback_text::VARCHAR AS FEEDBACK_TEXT,
    TRY_TO_TIMESTAMP_TZ(RAW_PAYLOAD:created_at::VARCHAR) AS CREATED_AT,
    TRY_TO_TIMESTAMP_TZ(RAW_PAYLOAD:updated_at::VARCHAR) AS UPDATED_AT,
    RAW_PAYLOAD:language::VARCHAR AS LANGUAGE,
    SHA2(RAW_PAYLOAD:feedback_text::VARCHAR, 256) AS FEEDBACK_HASH,
    SOURCE_FILE, BATCH_ID
  FROM ABC_RETAIL.BRONZE.BRONZE_CUSTOMER_FEEDBACK
  WHERE IS_PARSE_VALID
    AND RAW_PAYLOAD:feedback_id IS NOT NULL
    AND NULLIF(TRIM(RAW_PAYLOAD:feedback_text::VARCHAR), '') IS NOT NULL
  QUALIFY ROW_NUMBER() OVER (
    PARTITION BY RAW_PAYLOAD:feedback_id::VARCHAR
    ORDER BY TRY_TO_TIMESTAMP_TZ(RAW_PAYLOAD:updated_at::VARCHAR) DESC,
             INGEST_TS DESC
  ) = 1
) s
ON t.FEEDBACK_ID = s.FEEDBACK_ID
WHEN MATCHED AND s.FEEDBACK_HASH <> t.FEEDBACK_HASH THEN UPDATE SET
  FEEDBACK_TEXT = s.FEEDBACK_TEXT,
  UPDATED_AT = s.UPDATED_AT,
  FEEDBACK_HASH = s.FEEDBACK_HASH,
  SOURCE_FILE = s.SOURCE_FILE,
  BATCH_ID = s.BATCH_ID
WHEN NOT MATCHED THEN INSERT (
  FEEDBACK_ID, USER_ID, ORDER_ID, FEEDBACK_TEXT, CREATED_AT,
  UPDATED_AT, LANGUAGE, FEEDBACK_HASH, SOURCE_FILE, BATCH_ID
) VALUES (
  s.FEEDBACK_ID, s.USER_ID, s.ORDER_ID, s.FEEDBACK_TEXT, s.CREATED_AT,
  s.UPDATED_AT, s.LANGUAGE, s.FEEDBACK_HASH, s.SOURCE_FILE, s.BATCH_ID
);
```

Voi dbt, chuyen logic `MERGE` tren thanh incremental model co `unique_key`. Khai bao contract/test trong `models/staging/staging.yml`:

```
version: 2
models:
  - name: stg_orders
    description: "Current typed order state, SCD1 by updated_at"
    columns:
      - name: order_id
        data_tests: [not_null, unique]
      - name: status
        data_tests:
          - accepted_values:
              arguments:
                values: [created, paid, shipped, delivered, cancelled]

  - name: stg_customer_feedback
    columns:
      - name: feedback_id
        data_tests: [not_null, unique]
      - name: feedback_text
        data_tests: [not_null]
```

Chay rieng Silver va tat ca downstream:

```
dbt build --select tag:silver+
```

10. CDC trên Snowflake: Streams

Day la lab de hieu Snowflake CDC. Trong pipeline dbt chinh, incremental models doc Bronze bang `ingest_ts` lookback va `unique_key`; khong bat buoc consume Stream. Khong cho dbt va Snowflake Task cung consume mot Stream, vi consumer DML commit se tien offset va co the lam consumer con lai khong thay delta.

10.1. Hiểu đúng CDC

- Source CDC doc transaction log cua he thong nguon.
- File snapshots khong phai source CDC.
- Snowflake Stream luu offset va cho thay delta giua hai diem.
- Stream khong phai mot ban sao data; no la change tracking metadata.
- Offset chi tien khi stream duoc consume trong DML va transaction commit.

10.2. Tạo streams

```
CREATE OR REPLACE STREAM ABC_RETAIL.BRONZE.STR_ORDERS
  ON TABLE ABC_RETAIL.BRONZE.BRONZE_ORDERS
  APPEND_ONLY = TRUE;

CREATE OR REPLACE STREAM ABC_RETAIL.BRONZE.STR_CUSTOMERS
  ON TABLE ABC_RETAIL.BRONZE.BRONZE_CUSTOMERS
  APPEND_ONLY = TRUE;

CREATE OR REPLACE STREAM ABC_RETAIL.BRONZE.STR_CDC_DELETES
  ON TABLE ABC_RETAIL.BRONZE.BRONZE_CDC_DELETES
  APPEND_ONLY = TRUE;

CREATE OR REPLACE STREAM ABC_RETAIL.SILVER.STR_EVENTS_FOR_MART
  ON TABLE ABC_RETAIL.SILVER.SILVER_EVENTS;
```

Bronze append-only chi tao insert. Standard stream tren Silver co:

- `METADATA$ACTION`: INSERT hoac DELETE.
- `METADATA$ISUPDATE`: cho biet row action thuoc mot UPDATE.
- `METADATA$ROW_ID`: row identifier on dinh.

Kiem tra:

```
SELECT SYSTEM$STREAM_HAS_DATA('ABC_RETAIL.BRONZE.STR_ORDERS');
SELECT * FROM ABC_RETAIL.BRONZE.STR_ORDERS LIMIT 20;
SHOW STREAMS IN DATABASE ABC_RETAIL;
```

Moi consumer can stream rieng. Neu hai tasks cung doc mot stream bang DML, task commit truoc se dich offset va task sau co the khong con thay delta mong muon.

10.3. Source delete tombstones

`cdc_deletes` là source CDC mô phỏng có `operation='D'` và `source_lsn`. Không hard-delete Bronze. Consume tombstone trong transaction và áp dụng policy theo entity:

```
CREATE OR REPLACE TABLE ABC_RETAIL.GOVERNANCE.CDC_DELETE_AUDIT (
  ENTITY_TYPE VARCHAR,
  BUSINESS_KEY VARCHAR,
  OPERATION VARCHAR,
  CHANGED_AT TIMESTAMP_TZ,
  REASON VARCHAR,
  SOURCE_LSN VARCHAR,
  APPLIED_AT TIMESTAMP_TZ,
  RUN_ID VARCHAR
);

-- Ví dụ soft-delete customer dimension để bảo toàn historical facts.
UPDATE ABC_RETAIL.CORE_DWH.DIM_CUSTOMER d
SET IS_DELETED = TRUE,
    DELETED_AT = s.CHANGED_AT,
    UPDATED_RUN_ID = :RUN_ID
FROM (
  SELECT
    RAW_PAYLOAD:business_key::VARCHAR AS USER_ID,
    TRY_TO_TIMESTAMP_TZ(RAW_PAYLOAD:changed_at::VARCHAR) AS CHANGED_AT
  FROM ABC_RETAIL.BRONZE.STR_CDC_DELETES
  WHERE METADATA$ACTION = 'INSERT'
    AND RAW_PAYLOAD:entity_type::VARCHAR = 'customer'
```

```
    AND RAW_PAYLOAD:operation::VARCHAR = 'D'
) s
WHERE d.USER_ID = s.USER_ID
  AND d.IS_CURRENT;
```

Để chạy đoạn trên, thêm `IS_DELETED BOOLEAN DEFAULT FALSE` và `DELETED_AT TIMESTAMP_TZ` vào `DIM_CUSTOMER`. Với orders, thường không hard-delete facts đã báo cáo; dùng `IS_DELETED`, reversal record hoặc policy theo yêu cầu pháp lý.

11. Stored Procedure xử lý Orders

11.1. Chiến lược

Trong mot transaction:

1. Snapshot stream vao temporary table.
2. Parse va quarantine invalid rows.
3. Lay latest valid version cho moi order.
4. MERGE SCD1 vao current table.
5. Insert status history neu chua ton tai.
6. Rebuild latest items cho affected orders.
7. Commit va ghi audit.

Snapshot stream rat quan trong vi nhieu DML can doc cung mot delta. Snowflake DDL co the implicit commit, nen tao temp table truoc `BEGIN TRANSACTION`; ben trong transaction chi dung `INSERT`, `MERGE`, `DELETE` va `UPDATE`.

11.2. Procedure mẫu

```
CREATE OR REPLACE PROCEDURE ABC_RETAIL.SILVER.SP_PROCESS_ORDERS(RUN_ID VARCHAR)
RETURNS VARIANT
LANGUAGE SQL
EXECUTE AS OWNER
AS
$$
DECLARE
  V_ROWS_READ NUMBER DEFAULT 0;
  V_STARTED_AT TIMESTAMP_TZ DEFAULT CURRENT_TIMESTAMP();
BEGIN
  CREATE OR REPLACE TEMP TABLE TMP_ORDER_DELTA (
    RAW_PAYLOAD VARIANT,
    SOURCE_FILE VARCHAR,
    SOURCE_ROW_NUMBER NUMBER,
    BATCH_ID VARCHAR,
    INGEST_TS TIMESTAMP_TZ,
    PAYLOAD_HASH VARCHAR
  );

  CREATE OR REPLACE TEMP TABLE TMP_ORDER_PARSED (
    ORDER_ID VARCHAR,
    ORDER_TIME_UTC TIMESTAMP_TZ,
    USER_ID VARCHAR,
    ITEMS VARIANT,
    CURRENCY VARCHAR,
    STATUS VARCHAR,
    COUNTRY_CODE VARCHAR,
    UPDATED_AT TIMESTAMP_TZ,
    SOURCE_FILE VARCHAR,
    SOURCE_ROW_NUMBER NUMBER,
    BATCH_ID VARCHAR,
    INGEST_TS TIMESTAMP_TZ,
    RECORD_HASH VARCHAR,
    RAW_PAYLOAD VARIANT
  );

  CREATE OR REPLACE TEMP TABLE TMP_ORDER_LATEST LIKE TMP_ORDER_PARSED;

  BEGIN TRANSACTION;

  -- INSERT ... SELECT consumes/locks the stream inside this transaction.
  INSERT INTO TMP_ORDER_DELTA
  SELECT
    RAW_PAYLOAD,
    SOURCE_FILE,
    SOURCE_ROW_NUMBER,
    BATCH_ID,
    INGEST_TS,
    PAYLOAD_HASH
  FROM ABC_RETAIL.BRONZE.STR_ORDERS
  WHERE METADATA$ACTION = 'INSERT'
    AND IS_PARSE_VALID;

  SELECT COUNT(*) INTO :V_ROWS_READ FROM TMP_ORDER_DELTA;

  INSERT INTO TMP_ORDER_PARSED
  SELECT
    RAW_PAYLOAD:order_id::VARCHAR AS ORDER_ID,
    TRY_TO_TIMESTAMP_TZ(RAW_PAYLOAD:order_time::VARCHAR) AS ORDER_TIME_UTC,
    RAW_PAYLOAD:user_id::VARCHAR AS USER_ID,
    RAW_PAYLOAD:items AS ITEMS,
    UPPER(RAW_PAYLOAD:currency::VARCHAR) AS CURRENCY,
    RAW_PAYLOAD:status::VARCHAR AS STATUS,
    UPPER(RAW_PAYLOAD:shipping_address:country::VARCHAR) AS COUNTRY_CODE,
    TRY_TO_TIMESTAMP_TZ(RAW_PAYLOAD:updated_at::VARCHAR) AS UPDATED_AT,
    SOURCE_FILE, SOURCE_ROW_NUMBER, BATCH_ID, INGEST_TS,
    PAYLOAD_HASH AS RECORD_HASH,
    RAW_PAYLOAD
  FROM TMP_ORDER_DELTA;

  INSERT INTO ABC_RETAIL.GOVERNANCE.QUARANTINE_RECORDS
    (SOURCE_NAME, SOURCE_FILE, SOURCE_ROW_NUMBER, BATCH_ID, RAW_PAYLOAD,
     ERROR_CODE, ERROR_MESSAGE, RUN_ID)
  SELECT
    'orders', SOURCE_FILE, SOURCE_ROW_NUMBER, BATCH_ID, RAW_PAYLOAD,
    CASE
      WHEN ORDER_ID IS NULL THEN 'MISSING_KEY'
      WHEN ORDER_TIME_UTC IS NULL OR UPDATED_AT IS NULL
        THEN 'INVALID_TIMESTAMP'
      WHEN CURRENCY NOT IN ('USD', 'VND', 'EUR', 'JPY')
        THEN 'UNSUPPORTED_CURRENCY'
      ELSE 'INVALID_ORDER'
    END,
    'Order failed validation',
    :RUN_ID
  FROM TMP_ORDER_PARSED
```

```
WHERE ORDER_ID IS NULL
   OR ORDER_TIME_UTC IS NULL
   OR UPDATED_AT IS NULL
   OR CURRENCY NOT IN ('USD', 'VND', 'EUR', 'JPY');

INSERT INTO TMP_ORDER_LATEST
SELECT *
FROM TMP_ORDER_PARSED
WHERE ORDER_ID IS NOT NULL
  AND ORDER_TIME_UTC IS NOT NULL
  AND UPDATED_AT IS NOT NULL
  AND CURRENCY IN ('USD', 'VND', 'EUR', 'JPY')
QUALIFY ROW_NUMBER() OVER (
  PARTITION BY ORDER_ID
  ORDER BY UPDATED_AT DESC, INGEST_TS DESC, SOURCE_FILE DESC
) = 1;

MERGE INTO ABC_RETAIL.SILVER.SILVER_ORDERS_CURRENT t
USING TMP_ORDER_LATEST s
ON t.ORDER_ID = s.ORDER_ID
WHEN MATCHED AND s.UPDATED_AT > t.UPDATED_AT THEN UPDATE SET
  ORDER_TIME_UTC = s.ORDER_TIME_UTC,
  USER_ID = s.USER_ID,
  CURRENCY = s.CURRENCY,
  STATUS = s.STATUS,
  COUNTRY_CODE = s.COUNTRY_CODE,
  UPDATED_AT = s.UPDATED_AT,
  SOURCE_FILE = s.SOURCE_FILE,
  BATCH_ID = s.BATCH_ID,
  RECORD_HASH = s.RECORD_HASH
WHEN NOT MATCHED THEN INSERT (
  ORDER_ID, ORDER_TIME_UTC, USER_ID, CURRENCY, STATUS, COUNTRY_CODE,
  UPDATED_AT, SOURCE_FILE, BATCH_ID, RECORD_HASH
) VALUES (
  s.ORDER_ID, s.ORDER_TIME_UTC, s.USER_ID, s.CURRENCY, s.STATUS,
  s.COUNTRY_CODE, s.UPDATED_AT, s.SOURCE_FILE, s.BATCH_ID, s.RECORD_HASH
);

INSERT INTO ABC_RETAIL.SILVER.SILVER_ORDER_STATUS_HISTORY
  (ORDER_ID, STATUS, STATUS_AT, BATCH_ID, RECORD_HASH)
SELECT ORDER_ID, STATUS, UPDATED_AT, BATCH_ID, RECORD_HASH
FROM TMP_ORDER_PARSED s
WHERE ORDER_ID IS NOT NULL AND UPDATED_AT IS NOT NULL
  AND NOT EXISTS (
    SELECT 1
    FROM ABC_RETAIL.SILVER.SILVER_ORDER_STATUS_HISTORY h
    WHERE h.ORDER_ID = s.ORDER_ID
      AND h.STATUS = s.STATUS
      AND h.STATUS_AT = s.UPDATED_AT
  );

DELETE FROM ABC_RETAIL.SILVER.SILVER_ORDER_ITEMS i
USING TMP_ORDER_LATEST s
WHERE i.ORDER_ID = s.ORDER_ID
  AND s.UPDATED_AT >= i.UPDATED_AT;

INSERT INTO ABC_RETAIL.SILVER.SILVER_ORDER_ITEMS
SELECT
  s.ORDER_ID,
  f.INDEX + 1 AS LINE_NUMBER,
  f.VALUE:sku::VARCHAR,
  TRY_TO_DECIMAL(f.VALUE:qty::VARCHAR, 18, 3),
  TRY_TO_DECIMAL(f.VALUE:unit_price::VARCHAR, 18, 4),
  s.CURRENCY,
  r.RATE,
  TRY_TO_DECIMAL(f.VALUE:qty::VARCHAR, 18, 3)
    * TRY_TO_DECIMAL(f.VALUE:unit_price::VARCHAR, 18, 4)
    * r.RATE,
  s.UPDATED_AT
FROM TMP_ORDER_LATEST s,
  LATERAL FLATTEN(INPUT => s.ITEMS) f
LEFT JOIN ABC_RETAIL.SILVER.SILVER_FX_RATES r
  ON r.RATE_DATE = TO_DATE(s.ORDER_TIME_UTC)
 AND r.SOURCE_CURRENCY = s.CURRENCY
 AND r.TARGET_CURRENCY = 'USD'
WHERE f.VALUE:sku IS NOT NULL
  AND TRY_TO_DECIMAL(f.VALUE:qty::VARCHAR, 18, 3) > 0
  AND TRY_TO_DECIMAL(f.VALUE:unit_price::VARCHAR, 18, 4) >= 0;

INSERT INTO ABC_RETAIL.GOVERNANCE.AUDIT_PIPELINE_RUNS
  (RUN_ID, PIPELINE_NAME, TASK_NAME, STARTED_AT, ENDED_AT,
   STATUS, ROWS_READ, QUERY_ID)
VALUES
  (:RUN_ID, 'ABC_RETAIL', 'PROCESS_ORDERS', :V_STARTED_AT,
   CURRENT_TIMESTAMP(), 'SUCCESS', :V_ROWS_READ, LAST_QUERY_ID());

COMMIT;
RETURN OBJECT_CONSTRUCT('status', 'SUCCESS', 'rows_read', V_ROWS_READ);
```

```
EXCEPTION
  WHEN OTHER THEN
    ROLLBACK;
    INSERT INTO ABC_RETAIL.GOVERNANCE.AUDIT_PIPELINE_RUNS
      (RUN_ID, PIPELINE_NAME, TASK_NAME, STARTED_AT, ENDED_AT,
       STATUS, ERROR_MESSAGE)
    VALUES
      (:RUN_ID, 'ABC_RETAIL', 'PROCESS_ORDERS', :V_STARTED_AT,
       CURRENT_TIMESTAMP(), 'FAILED', SQLERRM);
    RETURN OBJECT_CONSTRUCT('status', 'FAILED', 'error', SQLERRM);
END;
$$;
```

Trong production, tach FX missing va invalid line item vao quarantine truoc insert. Procedure mau tap trung cho thay transaction va stream snapshot pattern.

12. SCD Type 2 triệt để, kể cả late-arriving change

dbt snapshot la cach nhanh de hoc SCD2:

```
-- snapshots/customer_snapshot.sql
{% snapshot customer_snapshot %}
{{
  config(
    target_schema='SNAPSHOTS',
    unique_key='user_id',
    strategy='timestamp',
    updated_at='updated_at',
    hard_deletes='invalidate'
  )
}}
select user_id, email, phone, country, updated_at
from {{ ref('stg_customer_changes') }}
{% endsnapshot %}
```

Chay:

```
dbt snapshot
dbt build --select customer_snapshot+
```

Snapshot ghi nhan trang thai ma no quan sat tai moi lan chay. Neu mot thay doi co business timestamp cu den muon va phai chen vao giua timeline da co, snapshot thong thuong khong tu dong rebuild toan bo effective ranges. De dap ung de bai "late-arriving SCD2 triet de", dung timeline model ben duoi cho cac business key bi anh huong. Snapshot van duoc giu nhu lab so sanh.

12.1. Tại sao MERGE đơn giản chưa đủ

Pattern "dong current row, insert row moi" chi dung khi changes den theo thu tu. Neu version ngay 10 den sau version ngay 20:

```
Sai:  [01 .. 20) -> [20 .. infinity)
Can:  [01 .. 10) -> [10 .. 20) -> [20 .. infinity)
```

Can rebuild timeline cho cac business keys bi anh huong.

12.2. Key map để surrogate key ổn định

Neu xoa va insert lai timeline bang sequence, facts cu co the bi orphan. Tao map giua `(user_id, effective_from)` va surrogate key:

```
CREATE OR REPLACE SEQUENCE ABC_RETAIL.CORE_DWH.SEQ_CUSTOMER_KEY START = 1;

CREATE OR REPLACE TABLE ABC_RETAIL.CORE_DWH.DIM_CUSTOMER_KEY_MAP (
  USER_ID VARCHAR,
  EFFECTIVE_FROM TIMESTAMP_TZ,
  CUSTOMER_KEY NUMBER DEFAULT ABC_RETAIL.CORE_DWH.SEQ_CUSTOMER_KEY.NEXTVAL,
  PRIMARY KEY (USER_ID, EFFECTIVE_FROM)
);

CREATE OR REPLACE TABLE ABC_RETAIL.CORE_DWH.DIM_CUSTOMER (
  CUSTOMER_KEY NUMBER,
  USER_ID VARCHAR,
  EMAIL VARCHAR,
  PHONE VARCHAR,
```

```
  COUNTRY_CODE VARCHAR,
  EFFECTIVE_FROM TIMESTAMP_TZ,
  EFFECTIVE_TO TIMESTAMP_TZ,
  IS_CURRENT BOOLEAN,
  IS_DELETED BOOLEAN DEFAULT FALSE,
  DELETED_AT TIMESTAMP_TZ,
  RECORD_HASH VARCHAR,
  CREATED_RUN_ID VARCHAR,
  UPDATED_RUN_ID VARCHAR,
  PRIMARY KEY (CUSTOMER_KEY)
);
```

12.3. Rebuild affected timelines

```
-- 1. Snapshot customer stream thanh TMP_CUSTOMER_DELTA.
-- 2. Parse valid rows vao SILVER_CUSTOMER_CHANGES.
-- 3. Lay affected users.
CREATE OR REPLACE TEMP TABLE TMP_AFFECTED_CUSTOMERS AS
SELECT DISTINCT USER_ID
FROM TMP_CUSTOMER_DELTA_PARSED
WHERE USER_ID IS NOT NULL AND UPDATED_AT IS NOT NULL;

-- 4. Tao/reuse surrogate key cho moi version.
MERGE INTO ABC_RETAIL.CORE_DWH.DIM_CUSTOMER_KEY_MAP t
USING (
  SELECT DISTINCT USER_ID, UPDATED_AT AS EFFECTIVE_FROM
  FROM ABC_RETAIL.SILVER.SILVER_CUSTOMER_CHANGES c
  WHERE EXISTS (
    SELECT 1 FROM TMP_AFFECTED_CUSTOMERS a
    WHERE a.USER_ID = c.USER_ID
  )
) s
ON t.USER_ID = s.USER_ID
AND t.EFFECTIVE_FROM = s.EFFECTIVE_FROM
WHEN NOT MATCHED THEN
  INSERT (USER_ID, EFFECTIVE_FROM)
  VALUES (s.USER_ID, s.EFFECTIVE_FROM);

-- 5. Xay timeline da deduplicate.
CREATE OR REPLACE TEMP TABLE TMP_CUSTOMER_TIMELINE AS
WITH dedup AS (
  SELECT *
  FROM ABC_RETAIL.SILVER.SILVER_CUSTOMER_CHANGES c
  WHERE EXISTS (
    SELECT 1 FROM TMP_AFFECTED_CUSTOMERS a
    WHERE a.USER_ID = c.USER_ID
  )
  QUALIFY ROW_NUMBER() OVER (
    PARTITION BY USER_ID, UPDATED_AT
    ORDER BY INGEST_TS DESC, SOURCE_FILE DESC
  ) = 1
),
changes_only AS (
  SELECT *
  FROM dedup
  QUALIFY RECORD_HASH <> COALESCE(
    LAG(RECORD_HASH) OVER (PARTITION BY USER_ID ORDER BY UPDATED_AT),
    '<FIRST>'
  )
)
SELECT
  m.CUSTOMER_KEY,
  c.USER_ID,
  c.EMAIL,
  c.PHONE,
  c.COUNTRY_CODE,
  c.UPDATED_AT AS EFFECTIVE_FROM,
  LEAD(c.UPDATED_AT) OVER (
    PARTITION BY c.USER_ID ORDER BY c.UPDATED_AT
  ) AS EFFECTIVE_TO,
  IFF(
    LEAD(c.UPDATED_AT) OVER (
      PARTITION BY c.USER_ID ORDER BY c.UPDATED_AT
    ) IS NULL,
    TRUE, FALSE
  ) AS IS_CURRENT,
  c.RECORD_HASH
FROM changes_only c
JOIN ABC_RETAIL.CORE_DWH.DIM_CUSTOMER_KEY_MAP m
  ON m.USER_ID = c.USER_ID
 AND m.EFFECTIVE_FROM = c.UPDATED_AT;

-- 6. Thay timeline cua affected keys trong cung transaction.
DELETE FROM ABC_RETAIL.CORE_DWH.DIM_CUSTOMER d
USING TMP_AFFECTED_CUSTOMERS a
WHERE d.USER_ID = a.USER_ID;

INSERT INTO ABC_RETAIL.CORE_DWH.DIM_CUSTOMER (
  CUSTOMER_KEY, USER_ID, EMAIL, PHONE, COUNTRY_CODE,
  EFFECTIVE_FROM, EFFECTIVE_TO, IS_CURRENT, RECORD_HASH,
  CREATED_RUN_ID, UPDATED_RUN_ID
)
SELECT
  CUSTOMER_KEY, USER_ID, EMAIL, PHONE, COUNTRY_CODE,
  EFFECTIVE_FROM, EFFECTIVE_TO, IS_CURRENT, RECORD_HASH,
  :RUN_ID, :RUN_ID
FROM TMP_CUSTOMER_TIMELINE;
```

12.4. Test SCD2

Chi mot current row:

```
SELECT USER_ID
FROM ABC_RETAIL.CORE_DWH.DIM_CUSTOMER
WHERE IS_CURRENT
GROUP BY USER_ID
HAVING COUNT(*) <> 1;
```

Khong overlap:

```
SELECT a.USER_ID, a.CUSTOMER_KEY, b.CUSTOMER_KEY
FROM ABC_RETAIL.CORE_DWH.DIM_CUSTOMER a
JOIN ABC_RETAIL.CORE_DWH.DIM_CUSTOMER b
  ON a.USER_ID = b.USER_ID
 AND a.CUSTOMER_KEY < b.CUSTOMER_KEY
 AND a.EFFECTIVE_FROM < COALESCE(b.EFFECTIVE_TO, '9999-12-31'::TIMESTAMP_TZ)
 AND b.EFFECTIVE_FROM < COALESCE(a.EFFECTIVE_TO, '9999-12-31'::TIMESTAMP_TZ);
```

13. Dimensional modeling

Tu day tro di, moi dimension/fact/mart la dbt model. Quy tac:

- Dung `{{ ref('model_name') }}` thay cho hard-code table downstream.
- Dung `{{ source('bronze', '...') }}` chi tai boundary Bronze.
- Ghi grain trong YAML description.
- Dimension/fact lon dung incremental; mart nho co the dung table.
- Moi primary business grain co `unique` va `not_null` test.

13.1. Chọn grain trước khi viết DDL

Fact	Grain	Loai
FACT_ORDER_LINE	Mot order line tai order version duoc chon	Transaction
FACT_CUSTOMER_EVENT	Mot event_id	Transaction
FACT_ORDER_STATUS	Mot lan order doi status	Transaction
MART_REVENUE_DAILY	Ngay, country, category	Periodic aggregate

Khong tron grain trong cung fact. Neu order co cung SKU hai lan, `order_id + sku` khong du unique; dung `line_number`.

13.2. Star schema

```
                DIM_DATE
                   |
DIM_CUSTOMER -- FACT_ORDER_LINE -- DIM_PRODUCT
                   |
              DIM_GEOGRAPHY
```

`DIM_PRODUCT` chua `category_name` va `subcategory_name`, nen query BI it join.

```
CREATE OR REPLACE TABLE ABC_RETAIL.CORE_DWH.DIM_PRODUCT (
  PRODUCT_KEY NUMBER AUTOINCREMENT,
  SKU VARCHAR,
  PRODUCT_NAME VARCHAR,
  BRAND VARCHAR,
  SUBCATEGORY_CODE VARCHAR,
  SUBCATEGORY_NAME VARCHAR,
  CATEGORY_CODE VARCHAR,
  CATEGORY_NAME VARCHAR,
  UNIT_COST NUMBER(18,4),
  IS_CURRENT BOOLEAN DEFAULT TRUE,
  PRIMARY KEY (PRODUCT_KEY)
);
```

13.3. Snowflake schema

```
FACT_ORDER_LINE
       |
 DIM_PRODUCT
       |
DIM_SUBCATEGORY
       |
 DIM_CATEGORY
```

```
CREATE OR REPLACE TABLE ABC_RETAIL.SANDBOX.DIM_CATEGORY_NORM (
  CATEGORY_KEY NUMBER AUTOINCREMENT,
  CATEGORY_CODE VARCHAR,
  CATEGORY_NAME VARCHAR
);

CREATE OR REPLACE TABLE ABC_RETAIL.SANDBOX.DIM_SUBCATEGORY_NORM (
  SUBCATEGORY_KEY NUMBER AUTOINCREMENT,
  SUBCATEGORY_CODE VARCHAR,
  SUBCATEGORY_NAME VARCHAR,
  CATEGORY_KEY NUMBER
);

CREATE OR REPLACE TABLE ABC_RETAIL.SANDBOX.DIM_PRODUCT_NORM (
  PRODUCT_KEY NUMBER AUTOINCREMENT,
  SKU VARCHAR,
  PRODUCT_NAME VARCHAR,
  BRAND VARCHAR,
  SUBCATEGORY_KEY NUMBER
);
```

13.4. Normalization và denormalization

Normalization:

- Giam lap lai category.
- Cap nhat ten category mot noi.
- Nhieu join hon.
- Kho hon cho self-service BI.

Denormalization:

- Query don gian va de hieu.
- Thuong phu hop consumption layer.
- Lap lai attribute va can pipeline dam bao consistency.

Trong Snowflake, storage re va compute tach biet khong co nghia denormalize luon tot. Quyet dinh dua tren workload, governance va query profile.

14. Load facts với point-in-time dimension lookup

14.1. Fact Order Line

```
CREATE OR REPLACE TABLE ABC_RETAIL.CORE_DWH.FACT_ORDER_LINE (
  ORDER_ID VARCHAR,
  LINE_NUMBER NUMBER,
  ORDER_DATE_KEY NUMBER,
  CUSTOMER_KEY NUMBER,
  PRODUCT_KEY NUMBER,
  GEOGRAPHY_KEY NUMBER,
  QTY NUMBER(18,3),
  UNIT_PRICE_ORIGINAL NUMBER(18,4),
  LINE_AMOUNT_USD NUMBER(18,4),
  ORDER_STATUS VARCHAR,
  ORDER_TIME_UTC TIMESTAMP_TZ,
  UPDATED_AT TIMESTAMP_TZ,
  LOAD_RUN_ID VARCHAR,
  PRIMARY KEY (ORDER_ID, LINE_NUMBER)
);
```

Lookup dung customer version:

```
MERGE INTO ABC_RETAIL.CORE_DWH.FACT_ORDER_LINE t
USING (
  SELECT
    i.ORDER_ID,
    i.LINE_NUMBER,
    TO_NUMBER(TO_CHAR(o.ORDER_TIME_UTC, 'YYYYMMDD')) AS ORDER_DATE_KEY,
    c.CUSTOMER_KEY,
    p.PRODUCT_KEY,
    g.GEOGRAPHY_KEY,
    i.QTY,
    i.UNIT_PRICE_ORIGINAL,
    i.LINE_AMOUNT_USD,
    o.STATUS AS ORDER_STATUS,
    o.ORDER_TIME_UTC,
    o.UPDATED_AT,
    :RUN_ID AS LOAD_RUN_ID
  FROM ABC_RETAIL.SILVER.SILVER_ORDER_ITEMS i
  JOIN ABC_RETAIL.SILVER.SILVER_ORDERS_CURRENT o
    ON o.ORDER_ID = i.ORDER_ID
  LEFT JOIN ABC_RETAIL.CORE_DWH.DIM_CUSTOMER c
    ON c.USER_ID = o.USER_ID
   AND o.ORDER_TIME_UTC >= c.EFFECTIVE_FROM
   AND o.ORDER_TIME_UTC < COALESCE(
         c.EFFECTIVE_TO, '9999-12-31'::TIMESTAMP_TZ
       )
  LEFT JOIN ABC_RETAIL.CORE_DWH.DIM_PRODUCT p
    ON p.SKU = i.SKU AND p.IS_CURRENT
  LEFT JOIN ABC_RETAIL.CORE_DWH.DIM_GEOGRAPHY g
    ON g.COUNTRY_CODE = o.COUNTRY_CODE
) s
ON t.ORDER_ID = s.ORDER_ID AND t.LINE_NUMBER = s.LINE_NUMBER
WHEN MATCHED AND s.UPDATED_AT > t.UPDATED_AT THEN UPDATE SET
  ORDER_DATE_KEY = s.ORDER_DATE_KEY,
  CUSTOMER_KEY = s.CUSTOMER_KEY,
  PRODUCT_KEY = s.PRODUCT_KEY,
  GEOGRAPHY_KEY = s.GEOGRAPHY_KEY,
  QTY = s.QTY,
  UNIT_PRICE_ORIGINAL = s.UNIT_PRICE_ORIGINAL,
  LINE_AMOUNT_USD = s.LINE_AMOUNT_USD,
  ORDER_STATUS = s.ORDER_STATUS,
  UPDATED_AT = s.UPDATED_AT,
  LOAD_RUN_ID = s.LOAD_RUN_ID
WHEN NOT MATCHED THEN INSERT (
  ORDER_ID, LINE_NUMBER, ORDER_DATE_KEY, CUSTOMER_KEY, PRODUCT_KEY,
  GEOGRAPHY_KEY, QTY, UNIT_PRICE_ORIGINAL, LINE_AMOUNT_USD,
  ORDER_STATUS, ORDER_TIME_UTC, UPDATED_AT, LOAD_RUN_ID
) VALUES (
  s.ORDER_ID, s.LINE_NUMBER, s.ORDER_DATE_KEY, s.CUSTOMER_KEY, s.PRODUCT_KEY,
  s.GEOGRAPHY_KEY, s.QTY, s.UNIT_PRICE_ORIGINAL, s.LINE_AMOUNT_USD,
  s.ORDER_STATUS, s.ORDER_TIME_UTC, s.UPDATED_AT, s.LOAD_RUN_ID
);
```

Tao Unknown dimension member (`-1`) va `COALESCE` key neu business cho phep fact den truoc dimension. Sau do co task re-key orphan facts.

15. Gold data marts

15.1. Revenue Daily

```
CREATE OR REPLACE TABLE ABC_RETAIL.MART.MART_REVENUE_DAILY AS
SELECT
  TO_DATE(f.ORDER_TIME_UTC) AS REPORT_DATE,
  g.COUNTRY_CODE,
  p.CATEGORY_NAME,
  COUNT(DISTINCT f.ORDER_ID) AS ORDER_COUNT,
  SUM(f.QTY) AS UNITS_SOLD,
  SUM(IFF(f.ORDER_STATUS IN ('cancelled', 'returned'), 0,
          f.LINE_AMOUNT_USD)) AS NET_REVENUE_USD,
  SUM(f.LINE_AMOUNT_USD) AS GROSS_REVENUE_USD
FROM ABC_RETAIL.CORE_DWH.FACT_ORDER_LINE f
LEFT JOIN ABC_RETAIL.CORE_DWH.DIM_PRODUCT p
  ON p.PRODUCT_KEY = f.PRODUCT_KEY
LEFT JOIN ABC_RETAIL.CORE_DWH.DIM_GEOGRAPHY g
  ON g.GEOGRAPHY_KEY = f.GEOGRAPHY_KEY
GROUP BY 1, 2, 3;
```

Business rule tren chi la demo. Neu refund co partial amount, tao refund fact thay vi dat net revenue = 0 cho toan order.

15.2. Daily Funnel

```
CREATE OR REPLACE TABLE ABC_RETAIL.MART.MART_DAILY_FUNNEL AS
SELECT
  TO_DATE(e.EVENT_TIME_UTC) AS REPORT_DATE,
  e.DEVICE_OS,
  COUNT(DISTINCT IFF(e.EVENT_TYPE = 'view',
                     e.ANALYTIC_USER_ID, NULL)) AS VIEW_USERS,
  COUNT(DISTINCT IFF(e.EVENT_TYPE = 'add_to_cart',
                     e.ANALYTIC_USER_ID, NULL)) AS CART_USERS,
  COUNT(DISTINCT IFF(e.EVENT_TYPE = 'checkout',
                     e.ANALYTIC_USER_ID, NULL)) AS CHECKOUT_USERS,
  COUNT(DISTINCT IFF(e.EVENT_TYPE = 'purchase',
                     e.ANALYTIC_USER_ID, NULL)) AS PURCHASE_USERS,
  DIV0(PURCHASE_USERS, VIEW_USERS) AS VIEW_TO_PURCHASE_RATE
FROM ABC_RETAIL.SILVER.SILVER_EVENTS e
GROUP BY 1, 2;
```

Alias reuse co the phu thuoc SQL resolution; neu muon portable, boc aggregate trong CTE roi tinh rate o SELECT ngoai.

15.3. Customer 360

```
CREATE OR REPLACE TABLE ABC_RETAIL.MART.MART_CUSTOMER_360 AS
WITH order_metrics AS (
  SELECT
    c.USER_ID,
    COUNT(DISTINCT f.ORDER_ID) AS TOTAL_ORDERS,
    SUM(IFF(f.ORDER_STATUS NOT IN ('cancelled', 'returned'),
            f.LINE_AMOUNT_USD, 0)) AS LIFETIME_REVENUE_USD,
    MIN(f.ORDER_TIME_UTC) AS FIRST_PURCHASE_AT,
    MAX(f.ORDER_TIME_UTC) AS LAST_PURCHASE_AT
  FROM ABC_RETAIL.CORE_DWH.FACT_ORDER_LINE f
  JOIN ABC_RETAIL.CORE_DWH.DIM_CUSTOMER c
    ON c.CUSTOMER_KEY = f.CUSTOMER_KEY
  GROUP BY c.USER_ID
)
SELECT
  c.USER_ID, c.EMAIL, c.PHONE, c.COUNTRY_CODE,
  COALESCE(o.TOTAL_ORDERS, 0) AS TOTAL_ORDERS,
  COALESCE(o.LIFETIME_REVENUE_USD, 0) AS LIFETIME_REVENUE_USD,
  o.FIRST_PURCHASE_AT, o.LAST_PURCHASE_AT
FROM ABC_RETAIL.CORE_DWH.DIM_CUSTOMER c
LEFT JOIN order_metrics o ON o.USER_ID = c.USER_ID
WHERE c.IS_CURRENT;
```

16. Dynamic Tables

Dynamic Table phu hop transformation declarative. `TARGET_LAG` la freshness target, khong phai cron interval va khong phai guarantee tuyet doi.

```
CREATE OR REPLACE DYNAMIC TABLE ABC_RETAIL.MART.DT_REVENUE_DAILY
  TARGET_LAG = '15 minutes'
  WAREHOUSE = ABC_TRANSFORM_WH
  REFRESH_MODE = INCREMENTAL
AS
SELECT
  TO_DATE(f.ORDER_TIME_UTC) AS REPORT_DATE,
  p.CATEGORY_NAME,
  COUNT(DISTINCT f.ORDER_ID) AS ORDER_COUNT,
  SUM(f.LINE_AMOUNT_USD) AS REVENUE_USD
FROM ABC_RETAIL.CORE_DWH.FACT_ORDER_LINE f
LEFT JOIN ABC_RETAIL.CORE_DWH.DIM_PRODUCT p
  ON p.PRODUCT_KEY = f.PRODUCT_KEY
GROUP BY 1, 2;
```

Kiem tra mode Snowflake da chon:

```
SHOW DYNAMIC TABLES LIKE 'DT_REVENUE_DAILY'
  IN SCHEMA ABC_RETAIL.MART;

SELECT *
FROM TABLE(
  INFORMATION_SCHEMA.DYNAMIC_TABLE_REFRESH_HISTORY(
    NAME => 'ABC_RETAIL.MART.DT_REVENUE_DAILY'
  )
)
ORDER BY DATA_TIMESTAMP DESC;
```

Neu explicit `INCREMENTAL` ma query khong duoc ho tro, CREATE se fail. Trong prototype co the dung `AUTO`, nhung khi production nen lock mode de chi phi de du doan.

17. Tasks và Task Graph

17.0. Orchestration chính bằng dbt

Khong can Snowflake Task de bat dau. Mot production-like run:

```
dbt run-operation load_all_bronze --args '{batch_id: scheduled}'
dbt build
dbt source freshness
```

Dat ba lenh trong dbt Cloud job, Airflow DAG, GitHub Actions schedule hoac orchestrator cua cong ty. Thu tu model duoc dbt suy ra tu `ref()`. Artifacts `manifest.json`, `run_results.json` va `sources.json` duoc luu de audit.

Phan 17.1-17.3 la lab de hoc Snowflake-native Tasks/Task Graph. Chon **mot** orchestrator lam owner cua production schedule; khong de dbt job va Task graph cung ghi vao mot target trong cung thoi diem.

17.1. Root scheduled task

```
CREATE OR REPLACE TASK ABC_RETAIL.GOVERNANCE.TASK_ROOT
  WAREHOUSE = ABC_TRANSFORM_WH
  SCHEDULE = 'USING CRON 0 * * * * UTC'
  AS
    SELECT 'ABC Retail hourly pipeline';
```

17.2. Child tasks

```
CREATE OR REPLACE TASK ABC_RETAIL.GOVERNANCE.TASK_PROCESS_ORDERS
  WAREHOUSE = ABC_TRANSFORM_WH
  AFTER ABC_RETAIL.GOVERNANCE.TASK_ROOT
  WHEN SYSTEM$STREAM_HAS_DATA('ABC_RETAIL.BRONZE.STR_ORDERS')
  AS
    CALL ABC_RETAIL.SILVER.SP_PROCESS_ORDERS(UUID_STRING());

CREATE OR REPLACE TASK ABC_RETAIL.GOVERNANCE.TASK_PROCESS_CUSTOMERS
  WAREHOUSE = ABC_TRANSFORM_WH
  AFTER ABC_RETAIL.GOVERNANCE.TASK_ROOT
  WHEN SYSTEM$STREAM_HAS_DATA('ABC_RETAIL.BRONZE.STR_CUSTOMERS')
  AS
    CALL ABC_RETAIL.SILVER.SP_PROCESS_CUSTOMERS_SCD2(UUID_STRING());

CREATE OR REPLACE TASK ABC_RETAIL.GOVERNANCE.TASK_BUILD_FACTS
  WAREHOUSE = ABC_TRANSFORM_WH
  AFTER ABC_RETAIL.GOVERNANCE.TASK_PROCESS_ORDERS,
        ABC_RETAIL.GOVERNANCE.TASK_PROCESS_CUSTOMERS
  AS
    CALL ABC_RETAIL.CORE_DWH.SP_BUILD_FACTS(UUID_STRING());
```

Task duoc tao o trang thai suspended. Resume root sau khi children san sang:

```
SELECT SYSTEM$TASK_DEPENDENTS_ENABLE(
  'ABC_RETAIL.GOVERNANCE.TASK_ROOT'
);

EXECUTE TASK ABC_RETAIL.GOVERNANCE.TASK_ROOT;
```

Theo doi:

```
SELECT *
FROM TABLE(
  ABC_RETAIL.INFORMATION_SCHEMA.TASK_HISTORY(
    SCHEDULED_TIME_RANGE_START => DATEADD('hour', -24, CURRENT_TIMESTAMP()),
    RESULT_LIMIT => 100
  )
)
ORDER BY SCHEDULED_TIME DESC;
```

17.3. Triggered task thay cron

Khi data den khong deu, co the tao task khong co `SCHEDULE`, chi co `WHEN SYSTEM$STREAM_HAS_DATA`. Triggered task giam polling va latency. Chon mot trong hai kieu cho cung mot step, tranh hai tasks tranh nhau consume mot stream.

18. Test và reconciliation

18.1. Idempotency

```
CREATE OR REPLACE TEMP TABLE BEFORE_COUNTS AS
SELECT
  (SELECT COUNT(*) FROM ABC_RETAIL.SILVER.SILVER_ORDERS_CURRENT) AS ORDERS,
  (SELECT COUNT(*) FROM ABC_RETAIL.CORE_DWH.FACT_ORDER_LINE) AS LINES;

-- Replay cung batch theo quy trinh test.
CALL ABC_RETAIL.SILVER.SP_PROCESS_ORDERS('IDEMPOTENCY-REPLAY');

SELECT
  b.ORDERS AS BEFORE_ORDERS,
  (SELECT COUNT(*) FROM ABC_RETAIL.SILVER.SILVER_ORDERS_CURRENT)
    AS AFTER_ORDERS,
  b.LINES AS BEFORE_LINES,
  (SELECT COUNT(*) FROM ABC_RETAIL.CORE_DWH.FACT_ORDER_LINE)
    AS AFTER_LINES
FROM BEFORE_COUNTS b;
```

18.2. Orphan fact

```
SELECT COUNT(*) AS ORPHAN_FACTS
FROM ABC_RETAIL.CORE_DWH.FACT_ORDER_LINE f
LEFT JOIN ABC_RETAIL.CORE_DWH.DIM_CUSTOMER c
  ON c.CUSTOMER_KEY = f.CUSTOMER_KEY
WHERE c.CUSTOMER_KEY IS NULL;
```

18.3. Revenue reconciliation

```
WITH fact_total AS (
  SELECT TO_DATE(ORDER_TIME_UTC) D, SUM(LINE_AMOUNT_USD) AMT
  FROM ABC_RETAIL.CORE_DWH.FACT_ORDER_LINE
  WHERE ORDER_STATUS NOT IN ('cancelled', 'returned')
  GROUP BY 1
),
mart_total AS (
  SELECT REPORT_DATE D, SUM(NET_REVENUE_USD) AMT
  FROM ABC_RETAIL.MART.MART_REVENUE_DAILY
  GROUP BY 1
)
SELECT
  COALESCE(f.D, m.D) AS REPORT_DATE,
  f.AMT AS FACT_AMOUNT,
  m.AMT AS MART_AMOUNT,
  COALESCE(f.AMT, 0) - COALESCE(m.AMT, 0) AS DIFF
FROM fact_total f
FULL OUTER JOIN mart_total m ON m.D = f.D
WHERE ABS(COALESCE(f.AMT, 0) - COALESCE(m.AMT, 0)) > 0.01;
```

Test thanh cong khi query tra zero rows.

19. Unload data

```
COPY INTO @ABC_RETAIL.LANDING.STG_OUTBOUND/revenue/report_date=
FROM (
  SELECT * FROM ABC_RETAIL.MART.MART_REVENUE_DAILY
)
FILE_FORMAT = (
  TYPE = PARQUET
  COMPRESSION = SNAPPY
)
HEADER = TRUE
OVERWRITE = TRUE
MAX_FILE_SIZE = 134217728;

LIST @ABC_RETAIL.LANDING.STG_OUTBOUND/revenue/;
```

CSV:

```
COPY INTO @ABC_RETAIL.LANDING.STG_OUTBOUND/revenue_csv/
FROM ABC_RETAIL.MART.MART_REVENUE_DAILY
FILE_FORMAT = (
  TYPE = CSV
  COMPRESSION = GZIP
  FIELD_OPTIONALLY_ENCLOSED_BY = '"'
  NULL_IF = ()
)
HEADER = TRUE
OVERWRITE = TRUE;
```

Download neu can:

```
snow stage copy \
  @ABC_RETAIL.LANDING.STG_OUTBOUND/revenue_csv \
  output/revenue_csv \
  --recursive \
  --connection abc_retail
```

20. Zero-Copy Cloning

Clone cho dev/test:

```
CREATE OR REPLACE DATABASE ABC_RETAIL_DEV CLONE ABC_RETAIL;

SELECT COUNT(*) FROM ABC_RETAIL.CORE_DWH.FACT_ORDER_LINE;
SELECT COUNT(*) FROM ABC_RETAIL_DEV.CORE_DWH.FACT_ORDER_LINE;

DELETE FROM ABC_RETAIL_DEV.CORE_DWH.FACT_ORDER_LINE
WHERE ORDER_ID = 'o001';

-- Source khong doi.
SELECT COUNT(*)
FROM ABC_RETAIL.CORE_DWH.FACT_ORDER_LINE
WHERE ORDER_ID = 'o001';
```

Clone ban dau share micro-partitions. Sau khi source hoac clone thay doi, cac micro-partitions moi phat sinh va storage tang. Grants co the khong duoc copy nhu mong doi, nen kiem tra `SHOW GRANTS`.

Historical clone:

```
CREATE OR REPLACE TABLE ABC_RETAIL.SANDBOX.FACT_ORDER_LINE_BEFORE_FIX
CLONE ABC_RETAIL.CORE_DWH.FACT_ORDER_LINE
BEFORE (STATEMENT => 'REPLACE_WITH_QUERY_ID');
```

21. Time Travel và Fail-safe

21.1. Query lịch sử

```
SELECT *
FROM ABC_RETAIL.CORE_DWH.FACT_ORDER_LINE
AT (OFFSET => -60 * 5)
WHERE ORDER_ID = 'o001';
```

21.2. Khôi phục update/delete sai

Thuc hien trong SANDBOX:

```
DELETE FROM ABC_RETAIL.SANDBOX.TEST_RECOVERY
WHERE ORDER_ID = 'o001';

SET BAD_QUERY_ID = LAST_QUERY_ID();

INSERT INTO ABC_RETAIL.SANDBOX.TEST_RECOVERY
SELECT *
FROM ABC_RETAIL.SANDBOX.TEST_RECOVERY
BEFORE (STATEMENT => $BAD_QUERY_ID)
WHERE ORDER_ID = 'o001';
```

Neu can khoi phuc toan table an toan, clone historical version, doi soat roi swap/merge thay vi ghi de truc tiep.

21.3. UNDROP

```
DROP TABLE ABC_RETAIL.SANDBOX.TEST_RECOVERY;
UNDROP TABLE ABC_RETAIL.SANDBOX.TEST_RECOVERY;
```

21.4. Fail-safe

- Time Travel la self-service trong retention window.
- Sau retention, permanent table data co the vao Fail-safe 7 ngay.
- Fail-safe la disaster recovery do Snowflake van hanh, khong phai backup de query hang ngay.
- Transient/temporary tables khong co Fail-safe; dung cho data co the tai tao.
- Time Travel va Fail-safe deu co storage implication.

22. Secure Data Sharing

22.1. Secure view

Khong share Customer 360 co email/phone. Chi share aggregate:

```
CREATE OR REPLACE SECURE VIEW ABC_RETAIL.MART.SV_PARTNER_REVENUE AS
SELECT
  REPORT_DATE,
  COUNTRY_CODE,
  CATEGORY_NAME,
  ORDER_COUNT,
  UNITS_SOLD,
  NET_REVENUE_USD
FROM ABC_RETAIL.MART.MART_REVENUE_DAILY;
```

22.2. Provider

```
USE ROLE ACCOUNTADMIN;

CREATE OR REPLACE SHARE ABC_RETAIL_PARTNER_SHARE
  COMMENT = 'Aggregated retail metrics';

GRANT USAGE ON DATABASE ABC_RETAIL
  TO SHARE ABC_RETAIL_PARTNER_SHARE;
GRANT USAGE ON SCHEMA ABC_RETAIL.MART
  TO SHARE ABC_RETAIL_PARTNER_SHARE;
GRANT SELECT ON VIEW ABC_RETAIL.MART.SV_PARTNER_REVENUE
  TO SHARE ABC_RETAIL_PARTNER_SHARE;

ALTER SHARE ABC_RETAIL_PARTNER_SHARE
  ADD ACCOUNTS = ORGNAME.CONSUMER_ACCOUNT;

SHOW GRANTS TO SHARE ABC_RETAIL_PARTNER_SHARE;
```

22.3. Consumer

```
CREATE DATABASE ABC_RETAIL_SHARED
FROM SHARE PROVIDER_ORG.PROVIDER_ACCOUNT.ABC_RETAIL_PARTNER_SHARE;

SELECT *
FROM ABC_RETAIL_SHARED.MART.SV_PARTNER_REVENUE;
```

Consumer chi doc va khong copy physical data. Neu khong co consumer account, dung `SIMULATED_DATA_SHARING_CONSUMER` de test secure view logic neu quyen va kieu view ho tro.

23. Cortex AI

23.1. Quyền

Grant chi cac functions can thiet:

```
USE ROLE ACCOUNTADMIN;

GRANT DATABASE ROLE SNOWFLAKE.CORTEX_USER TO ROLE ABC_ENGINEER;
GRANT USE AI FUNCTION AI_SENTIMENT ON ACCOUNT TO ROLE ABC_ENGINEER;
GRANT USE AI FUNCTION AI_CLASSIFY ON ACCOUNT TO ROLE ABC_ENGINEER;
GRANT USE AI FUNCTION AI_COMPLETE ON ACCOUNT TO ROLE ABC_ENGINEER;
```

Quyen va model allowlist co the thay doi theo account policy. Neu query bi chan, khong tang quyen tuy tien; xem error va lam viec voi account admin.

23.2. Enrichment table

```
CREATE OR REPLACE TABLE ABC_RETAIL.MART.MART_FEEDBACK_AI_ENRICHED (
  FEEDBACK_ID VARCHAR,
  FEEDBACK_HASH VARCHAR,
  SENTIMENT VARCHAR,
  CATEGORY VARIANT,
  SUMMARY VARCHAR,
  PROCESSED_AT TIMESTAMP_TZ,
  PROMPT_VERSION VARCHAR,
  AI_RUN_ID VARCHAR,
  PRIMARY KEY (FEEDBACK_ID)
);
```

23.3. Incremental AI processing

Dữ liệu bổ sung có một số email giả lập để kiểm thử PII handling. Chỉ gọi AI cho new/changed feedback và redact locally trước khi gửi text vào AI Functions:

```
MERGE INTO ABC_RETAIL.MART.MART_FEEDBACK_AI_ENRICHED t
USING (
  WITH candidates AS (
    SELECT
      f.*,
      REGEXP_REPLACE(
        REGEXP_REPLACE(
          f.FEEDBACK_TEXT,
          '[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+[.][A-Za-z]{2,}',
          '[EMAIL]'
        ),
        '[+]?[0-9][0-9(). x-]{7,}[0-9]',
        '[PHONE]'
      ) AS AI_TEXT
    FROM ABC_RETAIL.SILVER.SILVER_CUSTOMER_FEEDBACK f
    LEFT JOIN ABC_RETAIL.MART.MART_FEEDBACK_AI_ENRICHED old
      ON old.FEEDBACK_ID = f.FEEDBACK_ID
    WHERE old.FEEDBACK_ID IS NULL
       OR old.FEEDBACK_HASH <> SHA2(f.FEEDBACK_TEXT, 256)
  )
  SELECT
    FEEDBACK_ID,
    SHA2(FEEDBACK_TEXT, 256) AS FEEDBACK_HASH,
    AI_SENTIMENT(AI_TEXT) AS SENTIMENT,
    AI_CLASSIFY(
      AI_TEXT,
      ['delivery', 'product_quality', 'payment', 'support', 'other']
    ) AS CATEGORY,
    AI_COMPLETE(
      'mistral-large2',
      CONCAT(
        'Summarize this customer feedback in one sentence. ',
        'Do not infer personal data. Feedback: ',
        AI_TEXT
      )
    ) AS SUMMARY,
    CURRENT_TIMESTAMP() AS PROCESSED_AT,
    'feedback-v1' AS PROMPT_VERSION,
    UUID_STRING() AS AI_RUN_ID
  FROM candidates
) s
ON t.FEEDBACK_ID = s.FEEDBACK_ID
WHEN MATCHED THEN UPDATE SET
  FEEDBACK_HASH = s.FEEDBACK_HASH,
  SENTIMENT = s.SENTIMENT,
  CATEGORY = s.CATEGORY,
  SUMMARY = s.SUMMARY,
  PROCESSED_AT = s.PROCESSED_AT,
  PROMPT_VERSION = s.PROMPT_VERSION,
  AI_RUN_ID = s.AI_RUN_ID
WHEN NOT MATCHED THEN INSERT (
  FEEDBACK_ID, FEEDBACK_HASH, SENTIMENT, CATEGORY, SUMMARY,
  PROCESSED_AT, PROMPT_VERSION, AI_RUN_ID
) VALUES (
  s.FEEDBACK_ID, s.FEEDBACK_HASH, s.SENTIMENT, s.CATEGORY, s.SUMMARY,
```

```
  s.PROCESSED_AT, s.PROMPT_VERSION, s.AI_RUN_ID
);
```

Model name va regional availability co the thay doi. Chon model tu danh sach account cho phep thay vi hard-code production.

23.4. Đánh giá

Khong can Snowflake CLI. Copy file nhan vao dbt seed:

```
cp data/expected_results/customer_feedback_gold_labels.csv \
  abc_retail_dbt/seeds/customer_feedback_gold_labels.csv
cd abc_retail_dbt
dbt seed --select customer_feedback_gold_labels
```

Khai bao type/test trong `seeds/schema.yml`:

```
version: 2
seeds:
  - name: customer_feedback_gold_labels
    config:
      schema: GOVERNANCE
      column_types:
        feedback_id: varchar
        expected_sentiment: varchar
        expected_category: varchar
    columns:
      - name: feedback_id
        data_tests: [not_null, unique]
```

Sau do tinh accuracy trong singular test hoac audit query:

```
SELECT
  COUNT(*) AS TEST_CASES,
  AVG(IFF(e.SENTIMENT = g.EXPECTED_SENTIMENT, 1, 0))
    AS SENTIMENT_ACCURACY,
  AVG(IFF(e.CATEGORY:label::VARCHAR = g.EXPECTED_CATEGORY, 1, 0))
    AS CATEGORY_ACCURACY,
  COUNT_IF(e.SENTIMENT IS NULL OR e.CATEGORY IS NULL) / COUNT(*)
    AS NULL_RATE
FROM {{ ref('mart_feedback_ai_enriched') }} e
JOIN {{ ref('customer_feedback_gold_labels') }} g
  ON g.FEEDBACK_ID = e.FEEDBACK_ID;
```

Theo doi cost/usage:

```
SELECT *
FROM SNOWFLAKE.ACCOUNT_USAGE.CORTEX_FUNCTIONS_QUERY_USAGE_HISTORY
WHERE START_TIME >= DATEADD('day', -7, CURRENT_TIMESTAMP())
ORDER BY START_TIME DESC;
```

AI Functions tinh phi theo AI credits/token tuy function/model. Test voi tap nho truoc, hash input de tranh goi lai, va khong dua PII khong can thiet vao prompt.

24. Performance và cost

24.1. Query Profile

Trong Snowsight:

1. Chay query co `QUERY_TAG`.
2. Mo Query History.
3. Mo Query Profile.
4. Kiem tra bytes scanned, partitions scanned, spillage, join explosion va queue time.

Query metrics:

```
SELECT
  QUERY_ID,
  QUERY_TEXT,
  TOTAL_ELAPSED_TIME,
```

```
  BYTES_SCANNED,
  PARTITIONS_SCANNED,
  PARTITIONS_TOTAL,
  BYTES_SPILLED_TO_LOCAL_STORAGE,
  BYTES_SPILLED_TO_REMOTE_STORAGE
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE QUERY_TAG LIKE 'abc_retail_tutorial%'
  AND START_TIME >= DATEADD('day', -7, CURRENT_TIMESTAMP())
ORDER BY START_TIME DESC;
```

24.2. Micro-partition pruning

Snowflake tu dong tao micro-partitions. Khong partition thu cong nhu Spark. Filter tren cot co locality tot se giam partitions scanned.

Chi cluster khi table du lon va query pattern on dinh:

```
ALTER TABLE ABC_RETAIL.CORE_DWH.FACT_ORDER_LINE
  CLUSTER BY (TO_DATE(ORDER_TIME_UTC), CUSTOMER_KEY);

SELECT SYSTEM$CLUSTERING_INFORMATION(
  'ABC_RETAIL.CORE_DWH.FACT_ORDER_LINE',
  '(TO_DATE(ORDER_TIME_UTC), CUSTOMER_KEY)'
);
```

Voi du lieu demo nho, clustering co the ton credit ma khong co loi ich.

24.3. Cache-aware benchmark

- Result cache co the tra ket qua ma khong chay lai compute.
- Warehouse cache bi mat khi suspend.
- Khong so sanh hai design chi bang mot lan query.
- Ghi warehouse size, cache state, row count va query ID.
- Dung cung workload va toi thieu nhieu lan lap.

24.4. Cost guardrails

- X-Small cho demo.
- Auto-suspend 60 giay.
- Resume task chi khi test.
- Suspend task va warehouse sau demo.
- Cortex AI chi process changed feedback.
- Khong bat Search Optimization/Automatic Clustering neu chua co benchmark.
- Theo doi serverless/AI bang usage views va budgets.

25. Deployment và runbook

25.1. Thứ tự deploy

```
1. Snowsight: roles, warehouses, database and schemas
2. AWS + Snowsight: Storage Integration and S3 stage
3. dbt run-operation create_platform_objects
4. dbt debug and dbt parse
5. dbt run-operation load_all_bronze
6. dbt seed
7. dbt snapshot
8. dbt build
9. dbt source freshness
10. dbt docs generate
11. Optional labs: Streams, procedures, Tasks, Dynamic Tables
12. Snowsight: security policies, shares, recovery demos
```

25.2. Quy trình chạy một batch

1. Upstream ghi file moi vao dung S3 prefix trong `landing1310`.
2. `LIST` stage; refresh directory table neu dang dung manual refresh.
3. Chay `dbt run-operation load_all_bronze`.
4. Kiem tra copy history.
5. Chay `dbt build`.
6. Chay `dbt source freshness`.
7. Kiem tra `target/run_results.json` va audit.
8. Chay singular DQ/reconciliation tests.
9. Kiem tra mart freshness.
10. Publish dbt artifacts neu dung CI/CD.

25.3. Xử lý sự cố

Trieu chung	Nguyen nhan thuong gap	Xu ly
COPY bo qua file	File name da co trong load history	Dung ten/version moi; replay qua temp + MERGE
Stream rong	Da bi DML consume hoac chua co insert	Kiem tra task/audit va source table
Stream stale	Khong consume trong retention window	Recreate stream va backfill theo watermark
Task suspended	Moi tao hoac predecessor fail	Fix root cause, resume graph
SCD co 2 current rows	Transaction/rule timeline sai	Suspend pipeline, clone, rebuild affected keys
Fact orphan	Dimension den tre	Unknown member + re-key task
Dynamic Table full refresh	Query khong ho tro incremental	Rewrite query hoac chap nhan FULL
Cortex permission error	Role/model/region	Kiem tra grants va allowlist
Chi phi tang	Warehouse/task/AI lap lai	Suspend, kiem tra history, them guardrail

25.4. Recovery runbook

1. Suspend root task de dung ghi tiep.
2. Ghi lai query IDs va thoi diem su co.
3. Clone database hien tai de forensic.
4. Query `BEFORE(STATEMENT => query_id)`.
5. Doi soat historical state.
6. Merge/restore pham vi toi thieu.
7. Chay tests va reconciliation.
8. Resume task.
9. Ghi post-incident report.

26. Cleanup

Truoc khi cleanup, xac nhan khong can giu bang chung:

```
ALTER TASK ABC_RETAIL.GOVERNANCE.TASK_ROOT SUSPEND;
ALTER WAREHOUSE ABC_INGEST_WH SUSPEND;
ALTER WAREHOUSE ABC_TRANSFORM_WH SUSPEND;
ALTER WAREHOUSE ABC_BI_WH SUSPEND;
```

```
DROP SHARE IF EXISTS ABC_RETAIL_PARTNER_SHARE;
DROP DATABASE IF EXISTS ABC_RETAIL_DEV;
```

Chi drop database chinh khi da xuat evidence:

```
DROP DATABASE IF EXISTS ABC_RETAIL;
DROP WAREHOUSE IF EXISTS ABC_INGEST_WH;
DROP WAREHOUSE IF EXISTS ABC_TRANSFORM_WH;
DROP WAREHOUSE IF EXISTS ABC_BI_WH;
DROP RESOURCE MONITOR IF EXISTS ABC_MONTHLY_RM;
```

27. Checklist nghiệm thu

Foundation

- [ ] `dbt debug` va `dbt parse` thanh cong.
- [ ] Roles va grants duoc review.
- [ ] Warehouses auto-suspend.
- [ ] Storage Integration validate thanh cong.
- [ ] S3 external stage list duoc dung cac prefix trong `landing1310`.
- [ ] File formats doc dung JSONL, CSV va Parquet.

Pipeline

- [ ] Bronze append-only co lineage.
- [ ] Invalid rows vao quarantine.
- [ ] Orders SCD1 dung `updated_at`.
- [ ] Customer SCD2 xu ly late arrival.
- [ ] dbt incremental run khong bo sot late data.
- [ ] Neu lam lab Stream/Task, khong co consumer xung dot.
- [ ] Retry khong tao duplicate.

Modeling

- [ ] Grain cua moi fact duoc ghi ro.
- [ ] Fact lookup dung SCD version.
- [ ] Co Star va Snowflake variants.
- [ ] Co benchmark normalization/denormalization.

Platform

- [ ] Clone doc lap voi source.
- [ ] Time Travel recovery thanh cong.
- [ ] Fail-safe duoc giai thich dung.
- [ ] Secure view khong lo PII.
- [ ] AI co gold labels, accuracy va usage evidence.

Operations

- [ ] Audit khop dbt `run_results.json` va Snowflake query history.
- [ ] Reconciliation zero difference.
- [ ] Query profile duoc luu.

- [ ] Optional Tasks va warehouses duoc suspend sau demo.

28. Tài liệu tham khảo chính thức

1. Snowflake CLI installation:
https://docs.snowflake.com/en/developer-guide/snowflake-cli/installation/installation
2. Snowflake CLI connections:
https://docs.snowflake.com/en/developer-guide/snowflake-cli/connecting/configure-connections
3. Configure Storage Integration for Amazon S3:
https://docs.snowflake.com/en/user-guide/data-load-s3-config-storage-integration
4. Create an S3 external stage:
https://docs.snowflake.com/en/user-guide/data-load-s3-create-stage
5. CREATE STAGE reference:
https://docs.snowflake.com/en/sql-reference/sql/create-stage
6. SYSTEM$VALIDATE_STORAGE_INTEGRATION:
https://docs.snowflake.com/en/sql-reference/functions/system_validate_storage_integration
7. Snowpipe auto-ingest for Amazon S3:
https://docs.snowflake.com/en/user-guide/data-load-snowpipe-auto-s3
8. Unload data:
https://docs.snowflake.com/en/user-guide/data-unload-snowflake
9. CREATE STREAM:
https://docs.snowflake.com/en/sql-reference/sql/create-stream
10. Tasks:
https://docs.snowflake.com/en/user-guide/tasks-intro
11. Triggered Tasks:
https://docs.snowflake.com/en/user-guide/tasks-triggered
12. Dynamic Table target lag:
https://docs.snowflake.com/en/user-guide/dynamic-tables-target-lag
13. Zero-Copy Cloning:
https://docs.snowflake.com/en/sql-reference/sql/create-clone
14. Time Travel and Fail-safe:
https://docs.snowflake.com/en/user-guide/data-availability
15. Secure Data Sharing:
https://docs.snowflake.com/en/user-guide/data-sharing-intro
16. Secure views for sharing:
https://docs.snowflake.com/en/user-guide/data-sharing-secure-views
17. Cortex AI Functions:
https://docs.snowflake.com/en/user-guide/snowflake-cortex/aisql
18. Cortex AI privileges:
https://docs.snowflake.com/en/user-guide/snowflake-cortex/aisql-privileges-and-access

19. Cortex AI pricing and usage:
https://docs.snowflake.com/en/user-guide/snowflake-cortex/pricing
20. Resource Monitors:
https://docs.snowflake.com/en/user-guide/resource-monitors
21. dbt Quickstart for Snowflake:
https://docs.getdbt.com/guides/snowflake
22. dbt debug:
https://docs.getdbt.com/reference/commands/debug
23. dbt build:
https://docs.getdbt.com/reference/commands/build
24. dbt run-operation:
https://docs.getdbt.com/reference/commands/run-operation
25. Incremental models:
https://docs.getdbt.com/docs/build/incremental-models
26. dbt snapshots:
https://docs.getdbt.com/docs/build/snapshots
27. dbt data tests:
https://docs.getdbt.com/docs/build/data-tests

29. Lời kết

Project nay co gia tri khi ban co the giai thich moi quyet dinh:

- Vi sao Bronze append-only?
- Vi sao Stream khong dong nghia source CDC?
- Vi sao SCD2 can point-in-time lookup?
- Vi sao Gold duoc denormalize?
- Khi nao dung Task, khi nao dung Dynamic Table?
- Khi nao dung transient table va chap nhan khong co Fail-safe?
- Vi sao retry van an toan?
- Chi phi cua freshness va AI duoc kiem soat bang cach nao?

Neu code chay duoc nhung khong tra loi duoc cac cau hoi tren, project moi chi la demo. Neu pipeline co test, audit, recovery va design rationale, no da tro thanh mot capstone data engineering chuyen nghiep.